

Recursive Synthesis for Long-Horizon Terminal Tasks

Zhongzhi Li^{*1,2}, Yucheng Shi^{*†1}, Zongxia Li^{*1,3}, Ruhan Wang^{1,6}, Anhao Li⁵, Zixun Huang⁴, Junyao Yang^{1,7}, Lei Ke¹, Ninghao Liu⁸, Haitao Mi¹, Leowei Liang¹

¹Tencent HY LLM Frontier ²University of Georgia ³University of Maryland, College Park ⁴University of Pennsylvania ⁵University of Minnesota, Twin Cities ⁶Indiana University ⁷National University of Singapore ⁸Hong Kong Polytechnic University

^{*}Equal contribution, [†] Project lead.

z122754@uga.edu, zhongzhili@global.tencent.com



Hugging Face: Recursive Synthetic Terminal Tasks



Project Website

High-quality long-horizon training data for terminal agents is expensive to produce, often costing hundreds to thousands of dollars per task, because each task must keep the instruction, environment, reference solution, and verifier mutually consistent. Human authoring does not scale, and direct generation with large language models (LLMs) often breaks these dependencies. We present Recursive Synthetic Terminal Tasks (RST), a recursive verified synthesis framework for constructing long-horizon terminal-agent tasks at scale. Starting from verified seed tasks, RST extends the reference solution, realigns the verifier and instruction to the new workflow, validates the result in a fresh sandbox, and reuses accepted tasks as seeds for subsequent rounds. Across fifteen recursive rounds, RST produces 37,484 synthesized terminal-agent tasks at roughly \$0.05 per task. Task difficulty increases substantially over rounds: the median reference solution grows from 67 to 374 lines, the median number of executed commands grows from 40 to 244, and DeepSeek-V4-Pro pass@4 drops from 90% at R_1 to 2.5% at R_{15} . To demonstrate training utility, we collect rejection-sampled Qwen3.5 trajectories on the synthesized tasks and use them for supervised fine-tuning. Fine-tuning on these trajectories improves Qwen3.5-27B and Qwen3.5-122B-A10B by up to 10 points on Terminal-Bench 2, Terminal-Bench Hard, and Long-Horizon Terminal Bench, while agentic PPO lifts Qwen3.5-27B to 49.44%, 32.00%, and 22.07% on the three benchmarks, corresponding to relative gains of 20.0%, 41.2%, and 21.9% over the base model. Moreover, after 15 rounds, the recursion shows no ceiling: synthesis yield and validation rates remain stable as difficulty keeps climbing, indicating that the process can continue well beyond the scale reported here.

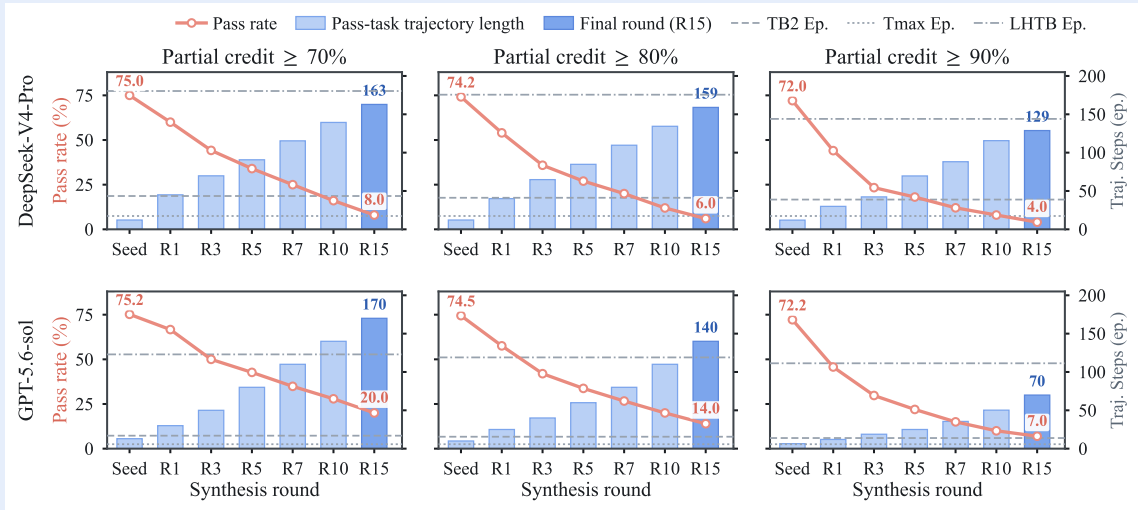


Figure 1 Pass rate and trajectory length across recursive synthesis rounds for DeepSeek-V4-Pro and GPT-5.6-sol under 70%, 80%, and 90% partial credit thresholds.

1 Introduction

Large language models (LLMs) are increasingly used as agents rather than one-shot generators. Instead of producing a single answer, they are expected to execute complex workflows and work toward a goal. This shift from simple chatting to multi-step tasks is reflected in recent work on tool use and software agents [2, 6, 17] such as interactive benchmarks for web, computer-use, and long-horizon problem solving [22, 24, 41]. In terminal-based tasks, agents must coordinate repository context, command-line interaction, and execution feedback over extended workflows [5, 23, 43]. As a result, progress depends not only on stronger models, but also on harness [11] and executable training data that teaches reliable long-horizon behavior [21].

However, high quality long-horizon tasks are hard to obtain at scale [34]. Useful training data for terminal agents must be verified through a runnable workspace with an instruction, meaningful execution feedback, and a reliable verifier to check success [11, 17, 23]. Human-written tasks and successful trajectories are expensive to produce, often ranging from hundreds to thousands of dollars per task [5, 36, 43]. Existing execution-based benchmarks and recent benchmark builders suggest that parts of data sourcing and environment construction can be automated [17, 35, 45], but they do not yet provide a scalable way to build *high quality* verified terminal-agent data.

In this paper, we propose RST, a recursive framework for synthesizing verifiable terminal-agent tasks. RST starts from an existing seed task, then extends its reference solution to create a longer and more demanding solution. It then updates the verifier and instruction to match the new solution and validates the complete task in a sandbox. Valid tasks are used to collect training trajectories and become seeds for the next synthesis round. We use DeepSeek-V4-Pro [4] to recursively produce increasingly difficult tasks and corresponding trajectories for agent training. To validate the usefulness of synthesized data, we conduct 15 rounds of recursive synthesis and produce 37,484 verified terminal tasks from 639 seed tasks. Later rounds require substantially more work: median solution length increases by $5.6\times$ and command use by $6.1\times$, while instruction length increases by only $1.4\times$. When the same solvers are evaluated across rounds, pass rates under the strictest criterion fall from 72% to 4% for DeepSeek-V4-Pro and 72.2% to 7% for GPT-5.6-sol at Seed to R_{15} . Trajectories collected by self-rolling out Qwen3.5 on these tasks improve Qwen3.5-27B and Qwen3.5-122B-A10B through supervised fine-tuning on Terminal-Bench 2, Terminal-Bench Hard, and Long-Horizon Terminal Bench. We also show that PPO training further increases Qwen3.5-27B by 8.24%, 9.33%, and 3.97% on the three benchmarks, respectively.

Our work shows the following insights:

1. **Scalable, low-cost synthesis of long-horizon tasks.** Our recursive, solution-first synthesis framework in which every accepted task carries an executable proof of solvability. From 639 seed tasks, fifteen rounds produce 37,484 verified terminal tasks at approximately \$0.05 per passed task, with no human authoring in the loop. Median solution length grows $5.6\times$ and command use $6.1\times$, and successful GPT-5.6-sol trajectories on late-round tasks exceed 100 steps.
2. **Consistent downstream improvement under standard training.** Self-collected trajectories by Qwen3.5 improve Qwen3.5-27B and Qwen3.5-122B-A10B on Terminal-Bench 2, Terminal-Bench Hard, and Long-Horizon Terminal Bench through plain supervised fine-tuning, with gains of up to 10 points.
3. **No observed ceiling.** After fifteen recursive rounds, passed-task yield and candidate pass rates remain stable, per-round structural growth stays positive, and domain, operator, and rewrite-family diversity are all preserved, even as solver pass rates fall from above 70% to 4–7%. The recursion shows no sign of saturation or collapse, and the pipeline depends on no particular seed domain or generation model, indicating that synthesis can continue to scale in both data quantity and quality.

2 Related Work

Terminal-agent benchmarks and long-horizon evaluation. Repository-level benchmarks such as SWE-Bench and Multi-SWE-Bench evaluate whether agents can modify codebases and satisfy executable tests [17, 45].

Table 1 Comparison of representative task datasets and environment-generation pipelines. Task and trajectory counts are reported separately because executable environments and agent interaction traces are distinct data units. Prior-work properties follow the corresponding papers and releases.

Dataset	Domain	Tasks [†]	Trajectories [‡]	Grounded?	Executable?	Adaptive?	RL-Validated?	Reseeding?
WizardLM [42]	General	–	250,000 ^a	✗	✗	✗	✗ (SFT)	✗
SWE-Gym [25]	SWE	2,438	N/R	✓	✓	✗	✗ (SFT)	✗
OpenThoughts-Agent-v1-SFT [29]	Terminal	N/R	15,209	△	✓	✗	✗ (SFT)	✗
TermiGen [52]	Terminal	3,500+	3,291	✗	✓	✗	✗ (SFT)	✗
RLVE-Gym [47]	Math/Algo	400	N/R	✓	✗	✓	✓ (DAPO)	✗
ScaleEnv [33]	Tool-Use	2,560	N/R	✗	✓	✗	✓ (GRPO)	✗
Endless Terminals [9]	Terminal	3,255	N/R	✗	✓	✗	✓ (PPO)	✗
Terminal-Corpus [27]	Terminal	N/R	254,000+	✓	✓	✗	✗ (SFT)	✗
TMax-15K [16]	Terminal	14,600	N/R ^b	✗	✓	✗	✓ (GRPO)	✗
SETA [30]	Terminal	4,567	1,112 ^c	✓	✓	✓	✓ (GRPO)	✗
RST (ours)	Terminal	37,484	327,189	✓	✓	✓	✓ (SFT, PPO)	✓

[†]Tasks. counts reusable task or environment instances rather than model executions. [‡]Trajectory counts collected agent interaction traces. N/R indicates that no fixed trajectory-corpus size is reported. ^aWizardLM reports single-turn instruction–response examples rather than terminal-agent trajectories. ^bTMax separately reports 16.5k SFT trajectories generated from an additional 2.2k-environment warm-start set. These are not counted as trajectories from TMax-15K. ^cSETA retains 1,112 successful SFT trajectories from 1,488 collected rollouts. △ indicates a mixture of grounded and synthetically generated sources.

SWE-Bench++ and SWE-Marathon extend this setting to broader and more demanding software-engineering tasks [5, 35]. Terminal-Bench evaluates interactive command-line workflows, while Harbor provides a common interface for sandboxed execution and verifier-based grading [11, 23]. Long-Horizon-Terminal-Bench extends this setting to stateful terminal tasks that require hundreds of interactions and uses dense verifier-based rewards to measure partial progress [21]. TerminalWorld constructs validated terminal tasks from real interaction records, including workflows that require more than fifty steps [3]. Broader suites evaluate agents across web, desktop, and multi-application settings [22, 24, 50]. Recent studies further emphasize persistent state, delayed feedback, and repeated tool use in long-horizon evaluation [18, 41]. Agents are commonly evaluated through shared scaffolds such as SWE-agent and OpenHands [36, 43]. These works primarily define evaluation settings, whereas RST recursively expands executable tasks and uses them to collect training trajectories.

Synthetic tasks and verifiable environments. Recent work generates tasks and environments for tool use, coding, and agent training [40, 51, 53]. Other systems reduce human authoring by scaling web, desktop, and general digital environments [7, 32, 37], while continual world generation and automatic environment construction extend this direction to evolving interaction settings [39, 47, 48]. For terminal agents, Endless Terminals and LiteCoder-Terminal construct executable training environments [9, 26], and CLI-Universe and SETA study their use in supervised and reinforcement learning [14, 30]. Among the compared methods, RST is the only one that repeatedly uses validated task bundles as seeds across multiple synthesis generations.

Recursive synthesis and agent self-improvement. Self-training and self-play improve reasoning models by generating new examples and filtering them with feedback [1, 12, 20, 31, 46]. Self-rewarding and verifier-guided methods extend this approach through learned or executable reward signals [13, 15, 19, 44]. However, recursive training can become unstable when filtering is weak or the reward is misspecified [8, 28, 49]. Several recent methods bring task evolution to agent settings. SETA adapts the difficulty and diversity of terminal environments [30]. BenchEvolver modifies executable solutions before deriving harder coding tasks and tests [38]. TRACE evolves agent tasks through validated and reproducible trajectories [10]. RST recursively evolves complete terminal tasks, including the workspace, reference solution, verifier, and public instruction. After each rewrite, these components are realigned and validated together. Accepted tasks are incorporated into subsequent synthesis seed pools and directly constitute the training task pool for verifier-based reinforcement learning. Successful rollouts collected on these tasks are retained as supervised fine-tuning trajectories.

3 Preliminaries

We formalize the executable terminal-task representation, the rollout protocol, and the criteria used to accept synthesized tasks. A terminal task is a self-contained executable problem. It contains the following components:

- `instruction.md`: the public task description;
- `task.toml`: runtime metadata and configuration;
- `environment/Dockerfile`: the initial environment and workspace;
- `solution/solve.sh`: the reference solution;
- `tests/test.sh` and `tests/test_state.py`: the private verifier.

Model interaction and grading are managed through Harbor using Terminus-2 as the agent harness [11]. For each rollout, Harbor creates an isolated sandbox and provides Terminus-2 with the public instruction and initialized workspace. Terminus-2 may inspect files, execute commands, and modify the workspace, but it cannot access the reference solution or private verifier. After the interaction ends, Harbor runs the verifier on the final workspace state. The verifier checks task outcomes rather than a fixed command sequence, allowing different valid solutions to receive credit.

A synthesized task is accepted only when it satisfies two conditions. First, the reference solution must pass the private verifier in a fresh sandbox; we refer to this as *oracle validity*. Second, every requirement checked by the verifier must be stated in the public instruction or inferable from the workspace; we refer to this as *contract validity*. The first condition establishes that the task is executable, while the second prevents private tests from introducing requirements hidden from the agent.

We denote the fifteen synthesis rounds by $R_1, \dots, R_{15}$. The process begins with 639 verified bootstrap tasks, and the bootstrap tasks themselves are not counted as a synthesis round. For each $r = 2, \dots, 15$, selected tasks from R_{r-1} are transformed and validated, and the accepted candidates form R_r . Accepted tasks may seed subsequent synthesis rounds and directly constitute the task pool for verifier-based reinforcement learning. Successful rollouts collected from these tasks are retained as supervised fine-tuning trajectories.

4 Method

RST constructs terminal-agent training tasks over multiple synthesis rounds. In each round, accepted tasks from the preceding round are selected as seeds. For each seed, the reference solution is extended, the verifier and public instruction are updated to match the new workflow, and the resulting task is validated in a fresh sandbox. This executable task format follows repository-level benchmarks [17, 35, 45] and terminal benchmarks [23]. Accepted tasks enter both the next synthesis seed pool and the reinforcement-learning task pool, while successful rollouts provide supervised fine-tuning trajectories.

Figure 2 summarizes the interaction between task synthesis and model training. Starting from a verified seed, the framework extends the reference solution and updates the verifier and public instruction to construct a harder candidate. The candidate is validated in a sandbox; unsolvable tasks are discarded, while accepted tasks enter the verified pool. This pool supplies seeds for subsequent synthesis rounds and tasks for verifier-based RL, while successful rollouts provide SFT trajectories.

Each synthesis round follows four stages. First, RST selects a feasible rewrite operator and defines the expected outcomes based on the seed task. It then extends the executable workflow and updates the solution, verifier, instruction, and environment to keep them consistent. Each candidate undergoes static checks, anti-shortcut and leakage audits, and validation in a fresh sandbox; recoverable failures receive bounded repair. Finally, accepted tasks are selected and cohort caps to form the next-round seed pool.¹

¹Prompt templates and implementation details are provided in Appendix B and Appendix C.

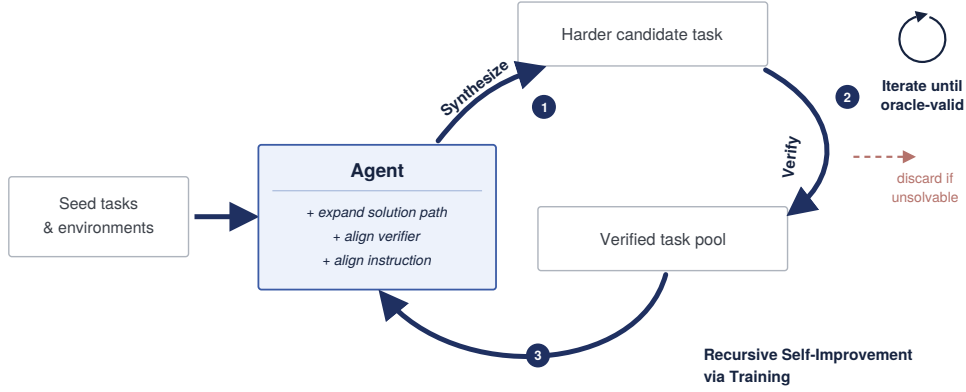


Figure 2 Recursive task synthesis and agent training in RST. Accepted tasks seed subsequent synthesis rounds and form the task pool for reinforcement learning, while successful rollouts provide supervised fine-tuning trajectories.

4.1 Seed Pool and Diversity-Capped Selection

We initialize the synthesis pipeline with 639 verified tasks as bootstrap seeds, sampled from TerminalWorld [3], a dataset of validated terminal tasks constructed from real interaction records. Applying the synthesis and validation process to these seeds produces 2,820 accepted tasks, which define R_1 ; the 639 bootstrap seeds are not counted as a synthesis round. For diversity analysis, the original category labels of the bootstrap seeds are consolidated into 19 domains. As shown in Figure 4, domain composition remains broadly distributed from the Seed pool through R_{15} : the largest domain remains below one quarter of the pool, normalized entropy changes only from 0.821 to 0.817, and the effective number of domains remains nearly unchanged (11.22 versus 11.09). These results show no evidence of broad domain collapse.²

For each subsequent round R_r , where $r = 2, \dots, 15$, seeds are selected from the accepted pool of R_{r-1} . Tasks missing required components are removed before selection. The remaining candidates are selected under caps on parent lineage, category, rewrite family, and generation cohort. These constraints prevent a small number of parents or rewrite patterns from dominating later rounds. The selected tasks enter the next synthesis round, and the accepted children form R_r .

4.2 Target Selection and Task Contract

Each synthesis round starts by selecting a suitable rewrite operator. The pipeline inspects the seed task, including its files, tools, dependencies, and existing workflow, to determine which extensions are feasible. It then selects one of the 40 operators shown in Figure 5. These operators are grouped into five families: Configuration and Control State; Data, Manifest, and Schema State; Filesystem and Resource Binding; Build, Cache, and Artifact State; and Runtime, Tooling, and Diagnostics.³

Before modifying the task, the generator records a rewrite plan. The plan defines the new required behavior, the corresponding changes to the reference solution, the intermediate and final outcomes checked by the verifier, and the information available to the agent through the instruction or workspace. It also identifies shortcuts that the verifier must reject. A plan is rejected if it introduces only cosmetic changes, adds checks unrelated to the executable workflow, or places required information only in private tests. An approved plan guides the subsequent updates to the solution, verifier, instruction, and environment.

4.3 Rewrite: Grow Solution, then Align

The rewrite proceeds from executable behavior and runtime conditions to the public task specification. First, the generator extends `solve.sh` with additional operations, such as inspecting files, deriving values, invoking

²The complete domain mapping is provided in Appendix A.

³Detailed operator definitions are provided in Appendix C.

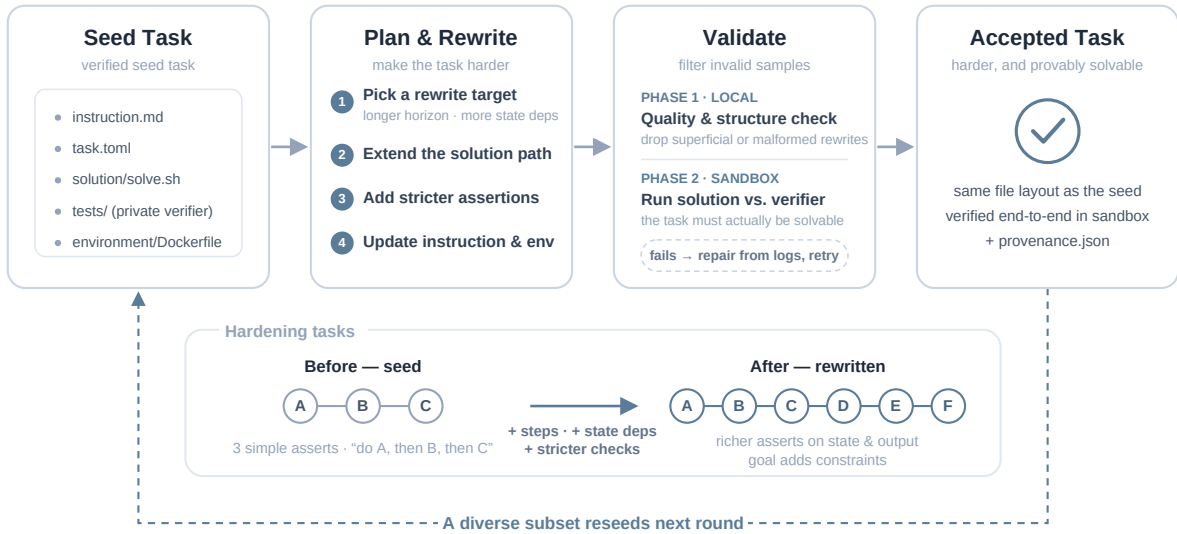


Figure 3 Detailed pipeline of one recursive synthesis round. A verified seed task undergoes target selection, staged rewriting, local and sandbox validation, and diversity-controlled reseeding.

tools, managing state, producing intermediate artifacts, or validating final outputs. The environment is then modified to support these operations. Such modifications may install new dependencies and CLI tools, provide additional files, fixtures, or configuration, initialize services and processes, or adjust permissions, paths, resource limits, and runtime settings. Once the solution and environment define a complete execution path, the verifier is updated to check the resulting artifacts and state transitions while rejecting placeholders, hard-coded outputs, and omitted intermediate work. The public instruction is revised to state the new objective and identify required information that is not discoverable from the environment. Task metadata is updated when the modified environment requires different resources, timeouts, or execution settings.

4.4 Validate: Local Filters and Sandbox Oracle

Validation proceeds in two stages. First, static checks reject candidates that are near-duplicates of their seeds, omit required files, contain invalid metadata, or expose private verifier details in the public instruction. This stage removes obvious failures before allocating a sandbox.

Candidates that pass the static checks are evaluated in a fresh sandbox. The environment is built from its initial state, the reference solution is executed, and the private verifier is run on the resulting workspace. When a failure is repairable, the validation logs are used for a limited number of repairs restricted to approved files. The candidate is then validated again; persistent failures are discarded. We use *oracle-passed* for candidates whose reference solution passes the private verifier in the sandbox. A candidate is *accepted* only after it also passes the instruction-verifier consistency checks.⁴

5 Experiments

We evaluate RST from two perspectives: the performance of the synthesis pipeline and the utility of the synthetic data for model training. The synthesis evaluation covers efficiency, structural growth, validity, diversity, and task difficulty across fifteen rounds. Efficiency is measured by normalized yield and generation cost; structural growth by changes in the solution and verifier; validity by instruction-verifier consistency; and diversity by rewrite-family balance, operator coverage, lineage retention, novelty, and near-duplicate

⁴The complete filters and repair procedure are provided in Appendix D.

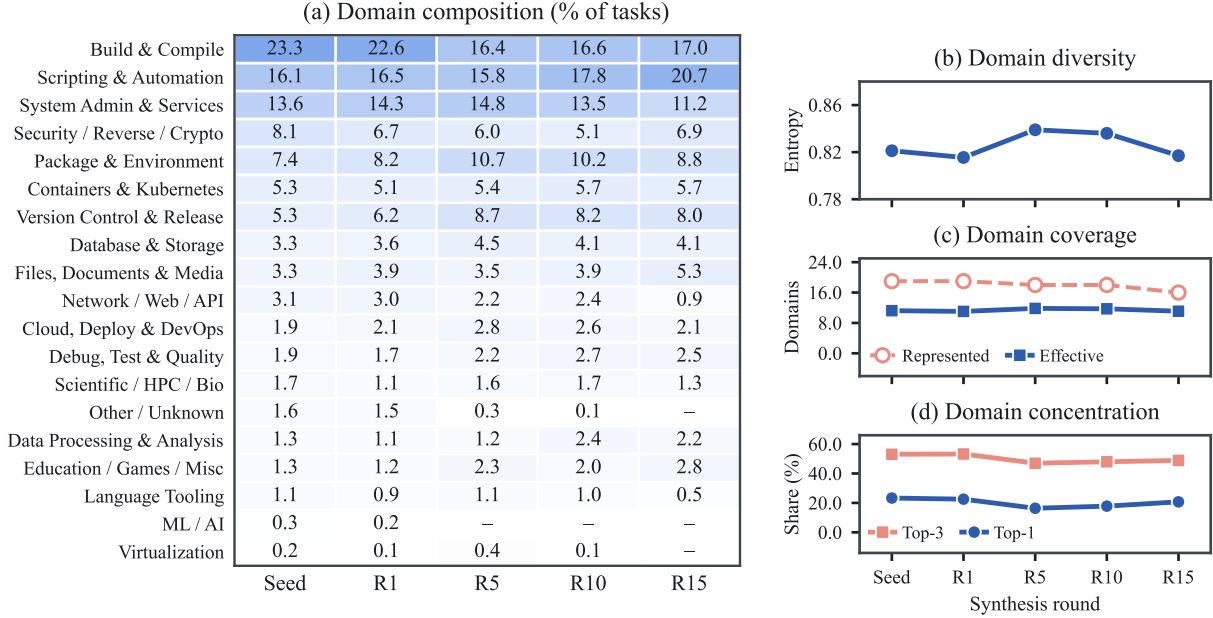


Figure 4 Domain composition and stability from the 639 bootstrap seeds through recursive synthesis. (a) Ancestral-domain proportions for the Seed pool and oracle-passed tasks at R_1 , R_5 , R_{10} , and R_{15} . (b) Distribution evenness measured by normalized Shannon entropy. (c) Domain breadth measured by the represented and effective numbers of domains. (d) Domain concentration measured by the proportions of tasks belonging to the Top-1 and Top-3 domains. The stable entropy and effective domain count indicate that domain diversity is preserved through R_{15} , with no broad domain collapse.

similarity. Task difficulty is evaluated on subsets from each round using DeepSeek-V4-Pro pass@4 and partial credit. Pass@4 measures full completion within four attempts, while partial credit is the fraction of verifier checks passed after a rollout. Training utility is evaluated through supervised fine-tuning of Qwen3.5-27B and Qwen3.5-122B-A10B and verifier-based reinforcement learning of Qwen3.5-27B. Terminal-Bench 2 evaluates broad terminal execution in standardized sandboxed environments with executable grading [23]. We evaluate transfer to an independently constructed task distribution using *Terminal-Bench Hard*, a 100-task subset drawn from TMax-15K [16]. Long-Horizon Terminal Bench evaluates persistent, multi-stage terminal workflows with many dependent interactions and dense partial-credit grading [21]. To verify that the observed transfer is not explained by benchmark leakage, we conduct a task-description audit comparing samples from R_1 , R_5 , R_{10} , and R_{15} with all 89 TB2 tasks, 100 Terminal-Bench Hard tasks, and 46 LHTB tasks (Table 2). Under the normalized 13-token sliding-window criterion, no benchmark task matches any sampled synthesis round, and the maximum pairwise 5-gram Jaccard similarity remains below 0.009. Moreover, unigram Jensen–Shannon divergence increases from R_1 to R_{15} for all three benchmarks, indicating that recursive synthesis produces an increasingly distinct task distribution rather than converging toward the benchmark distribution.

The synthesis process begins with 639 verified bootstrap tasks. Applying the synthesis and validation pipeline to these tasks produces 2,820 accepted tasks, which define R_1 ; the bootstrap tasks are not counted as a synthesis round. For each $r = 2, \dots, 15$, a subset of the accepted tasks in R_{r-1} is selected as the seed pool, and the accepted outputs of the new synthesis round form R_r . The resulting R_1 – R_{15} pools contain 37,484 tasks. Each submitted seed counts as one synthesis attempt, and synthesis yield is normalized per 1,000 attempts to support comparison across rounds. Structural, validity, and diversity statistics are computed from the accepted task pools, while solver difficulty is evaluated on matched subsets. All values are derived from the original generation and validation records.



Figure 5 Conceptual taxonomy of terminal-agent primitives used to characterize capability coverage. Ring labels are shortened; the implementation-level rewrite families and their 40 operators are defined separately in Appendix C.

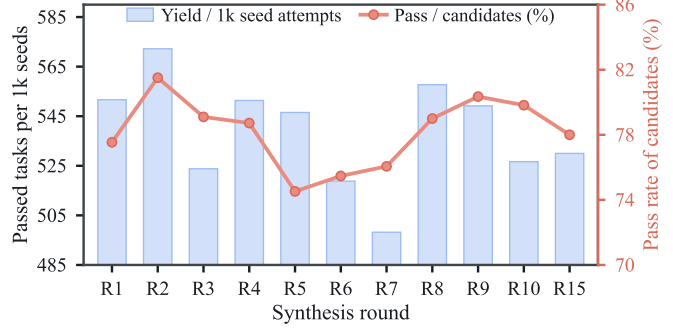


Figure 6 Passed-task yield per 1,000 seed attempts and candidate pass rate across recursive synthesis rounds. Both measures remain stable through R_{15} , with no systematic decline in synthesis throughput or validation success.

Table 2 Task-description distance and contamination analysis across recursive synthesis rounds. Unigram JSD measures the lexical-distribution difference between each round and the corresponding benchmark, while Max J_5 reports the largest pairwise 5-gram Jaccard similarity. Exact 13-token overlap reports the number of benchmark tasks containing at least one matching normalized 13-token window.

Round	Median tokens	Unigram JSD			Max J_5
		TB2	LHTB	TB Hard	
R_1	87.5	0.358	0.441	0.331	0.0081
R_5	104.0	0.387	0.461	0.359	0.0042
R_{10}	116.0	0.426	0.484	0.394	0.0028
R_{15}	121.5	0.433	0.485	0.406	0.0051
Exact 13-token overlap:		TB2 0/89	LHTB 0/46	TB Hard 0/100	

5.1 Recursive Task Synthesis Results

Recursive synthesis remains stable across fifteen rounds rather than collapsing under repeated reuse.

Figure 6 reports two measures of synthesis stability: passed-task yield per 1,000 seed attempts and candidate pass rate. Passed-task yield remains between 498.2 and 572.2 across rounds, reaching 530.0 in R_{15} compared with 551.6 in R_1 . Candidate pass rate likewise stays within a narrow range, from 74.5% to 81.5%, with similar values in R_1 and R_{15} (77.5% and 78.0%). These results show that the pipeline maintains comparable generation and validation performance even after fifteen rounds of recursive reuse.

At the same time, later-round tasks become substantially more complex in executable work rather than merely longer in prompt length.

Figures 7–9 show that from R_1 to R_{15} the median solution length grows from 67 to 374 lines, command count from 40 to 244, unique CLI tools from 17 to 71, control-flow operations from 6 to 45, file operations from 2 to 14, and verifier assertions from 17 to 57, while instruction length grows much more slowly from 85 to 122 words. The upper quantiles rise in parallel, indicating that this growth is distributed across the task population rather than driven by a small number of outliers. Figure 8 summarizes the same pattern as expansion factors, and Figure 9 confirms it at the task level: in R_{15} , median parent-child changes remain positive at 22 solution lines, 18 commands, and 3 verifier assertions, with positive deltas in

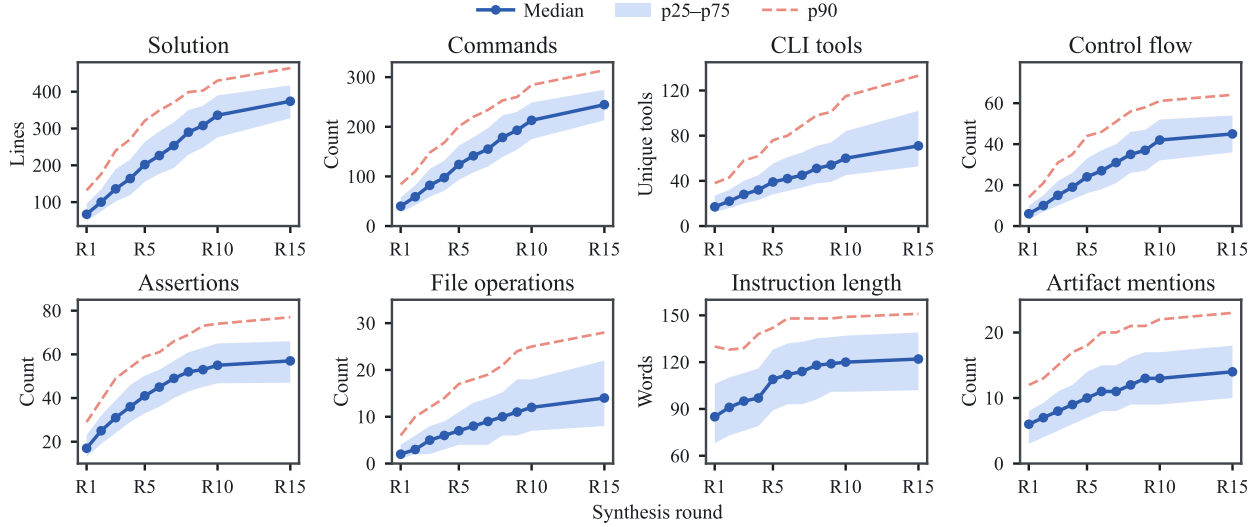


Figure 7 Quantile trends for eight task-structure metrics from R_1 to R_{15} . Median and upper-quantile values increase most strongly for solution length, command use, CLI tools, control flow, assertions, and file operations, while instruction length grows comparatively slowly.

78%, 77%, and 65% of pairs, respectively. Recursive synthesis therefore increases executable workload while continuing to add work at the individual task level.

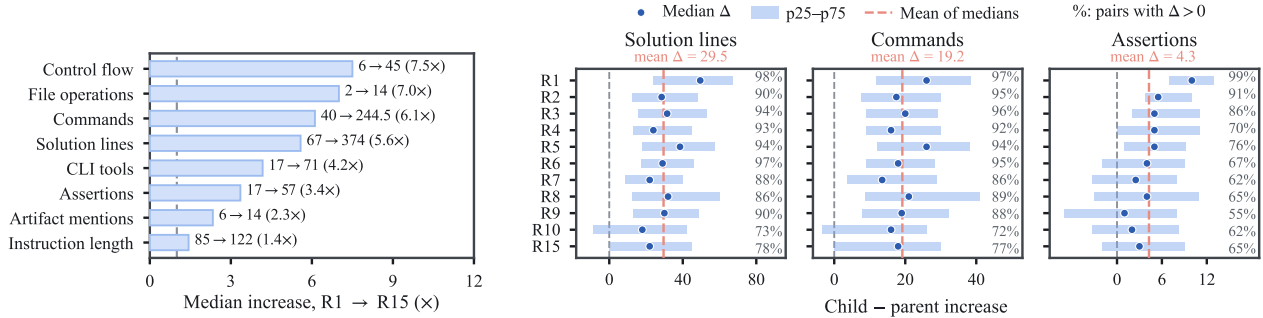


Figure 8 Median expansion factors from R_1 to R_{15} across task-structure metrics. Metrics associated with executable work increase substantially faster than instruction length.

Figure 9 Parent-child changes in solution length, command count, and verifier assertions from R_1 to R_{15} . Most accepted children exhibit positive changes relative to their immediate parents, indicating continued structural growth at the individual task level.

Later-round tasks are not only more complex but also better aligned with their public instructions.

Figures 10 and 11 show that hidden-check protection increases from 38.2% to 63.5%, while short-instruction risk decreases from 41.6% to 7.5%; references to private tests and literal leakage remain negligible, reaching only 0.1% and 0% in R_{15} , respectively. Median requirement coverage rises from 0.42 to 0.57, the share of weakly grounded tasks falls from 32.8% to 1.2%, and the strongly grounded share increases from 14.3% to 38.0%. Together, these results indicate that later tasks are better specified and that both binary success and partial-credit scores more faithfully reflect the public task contract.

5.2 Task Difficulty Analysis

To test whether the task-side growth observed above translates into agent-side difficulty, we evaluate DeepSeek-V4-Pro on matched task subsets from each synthesis round under fixed inference settings. Pass@4 measures full task completion within four attempts, while partial credit measures the fraction of verifier checks satisfied.

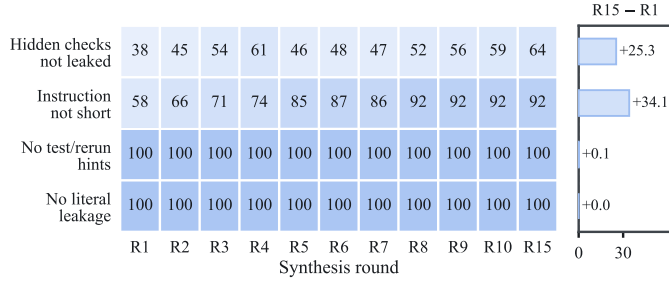


Figure 10 Public-instruction audit from R_1 to R_{15} . Hidden-check protection increases and short-instruction risk decreases, while references to private tests and literal leakage remain negligible.

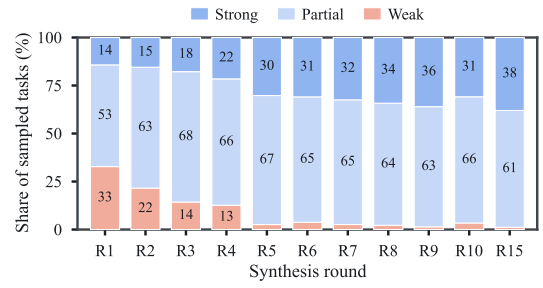


Figure 11 Requirement-to-verifier alignment from R_1 to R_{15} . Sampled tasks are grouped by the proportion of public requirements reflected in executable checks, providing a direct measure of instruction-verifier consistency.

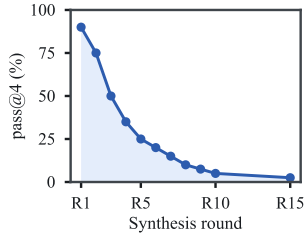


Figure 12 DeepSeek-V4-Pro pass@4 on task subsets across recursive synthesis rounds. The success decreases monotonically from 90% in R_1 to 2.5% in R_{15} .

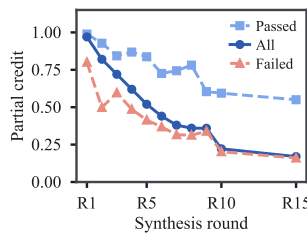


Figure 13 Mean verifier-based partial credit achieved by DeepSeek-V4-Pro, declining from 0.970 in R_1 to 0.170 in R_{15} .

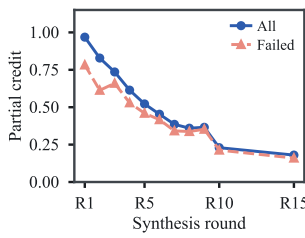


Figure 14 Attempt-level partial credit from R_1 to R_{15} , for credit distribution, with medians and p10-p90 shifting downward across rounds.

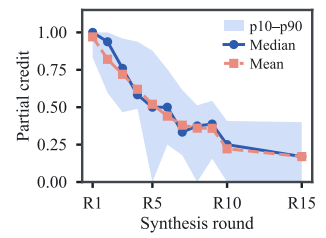


Figure 15 Task-level partial credit distribution, with medians and p10-p90 shifting downward across rounds.

Later-round tasks become substantially harder for a fixed solver. Figure 12 shows that DeepSeek-V4-Pro pass@4 declines monotonically from 90% in R_1 to 2.5% in R_{15} , a 36-fold reduction in full-task success. Figure 13 shows that mean partial credit falls in parallel, from 0.970 to 0.170. Because the solver and inference configuration remain unchanged across rounds, this decline reflects changes in the task distribution rather than changes in model capability. The results therefore show that the structural growth produced by recursive synthesis corresponds to genuinely greater task difficulty.

This increase in difficulty is broad rather than being driven by aggregation artifacts or a small number of outliers. Figure 14 shows that mean attempt-level partial credit decreases from 0.968 in R_1 to 0.180 in R_{15} , while the mean for failed attempts drops from 0.782 to 0.160, indicating that reduced progress appears consistently across individual rollouts. Figure 15 further shows that the entire task-level partial-credit distribution shifts downward: the median falls from 1.0 to 0.170, the p10 reaches 0, and the p90 declines to 0.400 by R_{15} . Increased difficulty is therefore distributed broadly across later-round tasks rather than concentrated in a narrow upper tail.

Later-round failures are also qualitatively harder, with far fewer near-misses and little recovery from repeated sampling. Figure 16 shows that failed attempts satisfying at least 75% of verifier checks fall from 86.4% in R_1 to 1.2% in R_{15} , while the share of tasks below half of the checks rises from 0% to 97.5%. Figure 17 shows that DeepSeek-V4-Pro solves at most 15% of tasks from R_7 onward, with pass@4 falling to 2.5% in R_{15} . Together, these results show that recursive synthesis does not merely reduce binary success rates; it produces later-round tasks on which the same solver makes substantially less partial progress and recovers far fewer complete solutions.

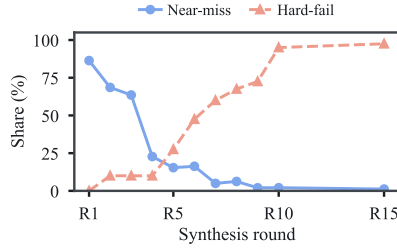


Figure 16 Distributional shift in partial-credit outcomes. Failed attempts above 0.75 partial credit fall from 86.4% to 1.2%, while tasks below 0.50 rise from 0% to 97.5%.

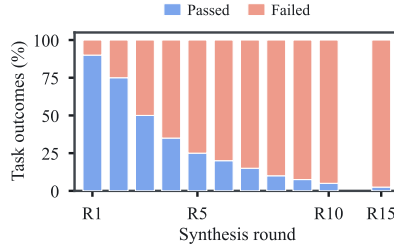


Figure 17 Task-level pass@4 outcomes for DeepSeek-V4-Pro. The pass rate decreases from 90% in R_1 to 20% in R_{15} .

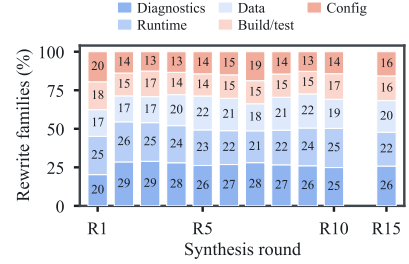


Figure 18 Distribution of accepted tasks across the five rewrite families. Entropy stays close to its maximum, and no family exceeds 29% in any round.

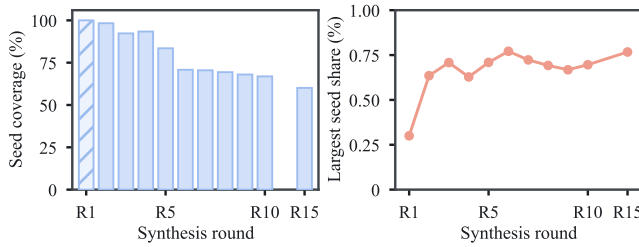


Figure 19 Coverage of bootstrap seeds across recursive rounds. R_{15} retains 218 of the 363 seeds represented in R_1 , and no single seed contributes more than 0.77% of the R_{15} tasks.

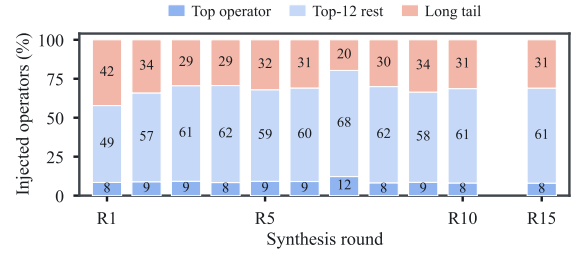


Figure 20 Rewrite-operator coverage from R_1 to R_{15} . Each round uses 31 to 38 of the 40 available operators, and the most frequent operator accounts for at most 12.2% of the tasks.

5.3 Data Diversity and Duplication Analysis

Recursive synthesis preserves broad diversity across domains and rewrite families rather than collapsing onto a single mode of task generation. Figure 4 shows that the task pool retains broad domain coverage across synthesis rounds, while Figure 18 shows that rewrite-family entropy remains between 2.26 and 2.31 bits, close to the maximum of $\log_2 5 = 2.32$ bits for five families. The largest family accounts for only 24.9% to 28.9% of accepted tasks across rounds, and the R_{15} distribution remains well spread across diagnostics and forensics, runtime substrate, data and artifact processing, build and test workflows, and configuration and state migration. The structural growth reported in Figures 7–9 therefore reflects multiple forms of task extension rather than one dominant rewrite pattern.

Repeated reseeding does not cause the dataset to become dominated by a small set of parents or operators. Figure 19 shows that bootstrap-seed coverage decreases gradually from 98.3% in R_2 to 60.1% in R_{15} , indicating that some seed sources are filtered out over time. However, no individual seed contributes more than 0.77% of the tasks in any round, so the remaining pool does not become concentrated around a small number of lineages. Figure 20 likewise shows that the pipeline continues to use a broad set of rewrite operations: 36 of the 40 operators remain represented in R_{15} , the most frequent operator accounts for only 8.0% of tasks, and operators outside the twelve most frequent still account for 31.0%. Recursive reuse therefore remains diverse both in ancestry and in transformation mechanism.

Although later-round tasks become somewhat more similar, recursive synthesis does not collapse into repeated copies. Figures 21 and 22 show that in R_{15} the median parent-child novelty remains 0.36 for the instruction, 0.18 for the solution, and 0.33 for the verifier, indicating that later children still modify all three task components even while retaining more implementation content from their parents. Within-round nearest-neighbor similarity increases from a median of 0.223 in R_1 to 0.464 in R_{15} , but the median remains

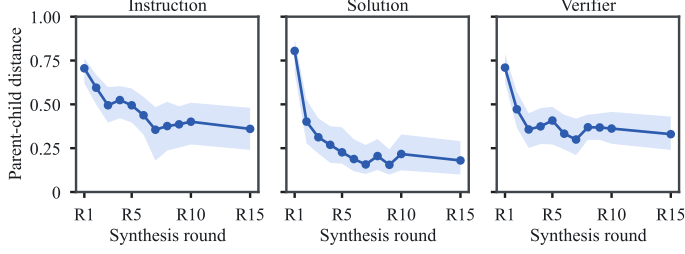


Figure 21 Token-level parent-child novelty in the instruction, solution, and verifier from R_1 to R_{15} . Later-round rewrites retain more content from their parents while continuing to modify all three task components.

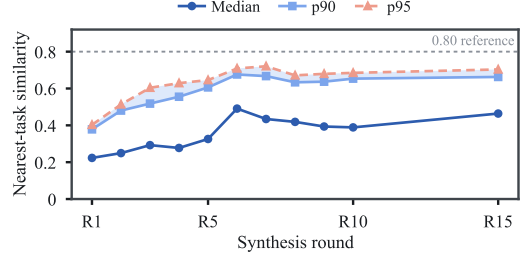


Figure 22 Within-round nearest-neighbor similarity from R_1 to R_{15} . The R_{15} median remains below 0.5, while p95 reaches 0.703, indicating broad task variation with a limited high-similarity tail.

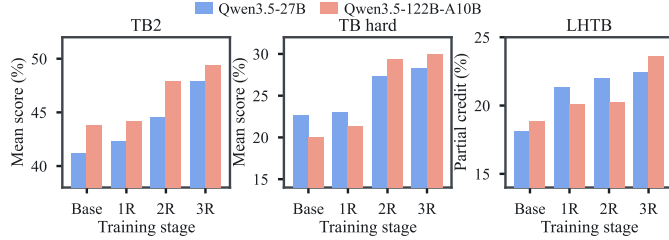


Figure 23 Benchmark performance of Qwen3.5-27B and Qwen3.5-122B-A10B after supervised fine-tuning on trajectories from progressively more synthesis rounds, reported on Terminal-Bench 2, Terminal-Bench Hard, and Long-Horizon Terminal Bench (mean partial credit).

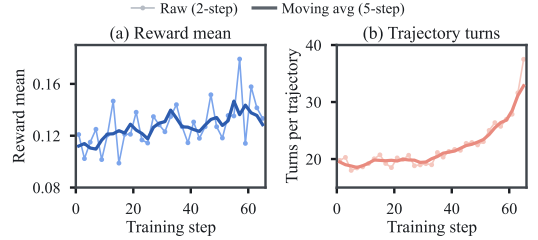


Figure 24 Reinforcement-learning dynamics on the synthesized terminal tasks: (a) mean verifier reward, rising from approximately 0.11 to above 0.14, and (b) mean interaction turns per trajectory, rising from 19–20 to more than 30 turns.

below 0.5, meaning that a typical R_{15} task still shares fewer than half of its combined instruction, solution, and verifier tokens with its closest neighbor. The p95 value of 0.703 shows that high similarity is concentrated in a limited upper tail rather than across the entire task pool. Taken together, these results show that recursive synthesis increases difficulty without collapsing into duplicates, while also highlighting a manageable high-similarity tail for future deduplication.

5.4 Supervised Fine-Tuning Results

The preceding analyses show that recursive synthesis produces increasingly difficult tasks while maintaining validation stability and broad data coverage. We next evaluate whether trajectories collected from these tasks improve terminal-agent performance. Successful Qwen3.5 rollouts are used to fine-tune Qwen3.5-27B and Qwen3.5-122B-A10B. The trained checkpoints are evaluated on Terminal-Bench 2 [23], Terminal-Bench Hard [16], and Long-Horizon Terminal Bench [21]. Each checkpoint is compared with its corresponding base model under the same evaluation configuration, allowing to measure both the overall training gain and the effect of incorporating additional recursive rounds.

Figure 23 and Table 3 compare the base models with checkpoints fine-tuned on trajectories from one, two, and three recursive synthesis stages. Performance increases monotonically for both model sizes on all three benchmarks. At the three-round stage, Terminal-Bench 2 improves from 41.2% to 47.9% for Qwen3.5-27B and from 43.8% to 49.4% for Qwen3.5-122B-A10B. Terminal-Bench Hard improves from 22.7% to 28.3% and from 20.0% to 30.0%, respectively. On Long-Horizon Terminal Bench, mean partial credit increases from 18.1% to 22.4% for Qwen3.5-27B and from 18.9% to 23.6% for Qwen3.5-122B-A10B.

All checkpoints are evaluated under the same benchmark configurations and compared with the corresponding base model. The consistent gains across training stages show that adding trajectories from further synthesis rounds continues to improve performance beyond the initial round. Improvements on both model sizes indicate

Table 3 Evaluation of Qwen3.5-27B and Qwen3.5-122B-A10B across successive training rounds on Terminal-Bench 2 (TB2), Terminal-Bench Hard, and the Long-Horizon Terminal Benchmark (LHTB). Base denotes the checkpoint before training, while Round 1–3 denote checkpoints obtained after successive training rounds. Each entry reports the evaluation mean $\pm$ standard deviation.

Model (Non-thinking Mode)	Training stage	Pass rate (% , mean $\pm$ std.)		
		Terminal-Bench 2	Terminal-Bench Hard	LHTB
Qwen3.5-27B	Base	41.20 $\pm$ 1.72	22.67 $\pm$ 2.52	18.10 $\pm$ 0.89
	Round 1	42.32 $\pm$ 4.68	23.00 $\pm$ 1.73	21.32 $\pm$ 0.86
	Round 2	44.57 $\pm$ 4.25	27.33 $\pm$ 1.53	21.99 $\pm$ 1.39
	Round 3	47.94 $\pm$ 2.34	28.33 $\pm$ 0.58	22.44 $\pm$ 0.40
Qwen3.5-122B-A10B	Base	43.82 $\pm$ 2.97	20.00 $\pm$ 1.00	18.85 $\pm$ 1.16
	Round 1	44.19 $\pm$ 1.30	21.33 $\pm$ 3.21	20.06 $\pm$ 0.32
	Round 2	47.94 $\pm$ 1.72	29.33 $\pm$ 2.08	20.24 $\pm$ 0.60
	Round 3	49.44 $\pm$ 1.12	30.00 $\pm$ 1.73	23.63 $\pm$ 2.76

that the benefit is not specific to a single model scale. Performance gains on Terminal-Bench 2, Terminal-Bench Hard, and Long-Horizon Terminal Bench demonstrate transfer across broad terminal execution, independently constructed tasks, and long-horizon interaction.

5.5 Terminal Agentic Reinforcement Learning Results

We then report our agentic reinforcement-learning dynamics on the synthesized tasks. The policy is initialized from the Qwen3.5-27B base checkpoint, corresponding to a cold actor start from the released base weights at rollout step zero. The PPO value head is warm-loaded from a prior terminal-agent critic checkpoint and receives two critic-only warm-up steps. The optimizer uses PPO with an actor-critic value function and generalized advantage estimation. The KL penalty and entropy bonus are disabled, PPO clipping uses $\epsilon = 0.2$, and advantages are whitened to zero mean and unit variance per batch.

The RL training pool is the synth-all set of 37,484 synthesized terminal-agent tasks, consisting of the R_1 synthesized pool and oracle-passed tasks from R_2 – R_{15} of the self-improvement loop. Rewrite variants are retained without deduplication. Each task is a self-contained terminal environment with task metadata, public instruction, verifier tests, reference solution, and Dockerfile, executed in a Daytona sandbox. The pool is reshuffled each epoch so that each batch mixes rounds and difficulty levels. Reward is computed from each task’s built-in verifier with customized reward shaping. Figure 24 shows raw measurements logged at two-step intervals together with a five-step moving average.

Figure 24(a) reports mean verifier reward during PPO training. Reward varies across updates because each batch contains tasks with different difficulty levels and numbers of verifier checks. Nevertheless, the five-step moving average increases from approximately 0.11 at the beginning of training to above 0.14 in later updates, with its highest values occurring around steps 55 to 60. The increase indicates that the policy satisfies more verifier checks as training progresses and that the synthesized tasks provide an effective graded learning signal. Figure 24(b) shows that mean trajectory length also increases during training. The five-step moving average remains near 19 to 20 turns during the early updates, begins rising after approximately step 30, and exceeds 30 turns in the later stage. This increase occurs alongside the rise in verifier reward, indicating that longer interactions are associated with greater task progress rather than an immediate decline in execution quality. The policy therefore learns to sustain longer terminal interactions while satisfying more verifier checks.

Table 5 Case study of one exact recursive lineage. The task keeps the same core objective, generating reliable JSON-diff regression outputs, while later rounds add configuration, failure diagnosis, release-note updates, and count reconciliation. The last column reports the number of non-empty lines in the reference `solution/solve.sh`.

Checkpoint	Public task	Added context	Agent work required	Lines
Seed	Generate six JSON-diff reports from fixed fixture pairs.	Manual CLI example with fixed input files.	Run <code>gendiff</code> on two JSON fixture pairs in three output formats.	10
R1	Generate all diff outputs specified by <code>tasks.json</code> .	The comparison matrix is moved into configuration.	Read <code>tasks.json</code> , batch-run all comparisons, and write a validation report.	66
R5	Repair <code>tasks.json</code> using <code>check_config.sh</code> , then regenerate the diffs.	The configuration may be invalid and must be diagnosed.	Run the checker, interpret failures, fix the task list, rerun the suite, and aggregate change counts.	219
R10	Use <code>CHANGELOG.md</code> to add the new nested-fixture regression case.	A release note introduces a new fixture and required comparison.	Update the regression matrix, generate the new output, and satisfy the local report validator.	264
R15	Repair inconsistent configuration and fixture data; reconcile expected diff counts.	Configuration, fixtures, expected counts, and tests must agree.	Fix <code>tasks.json</code> and JSON fixtures, regenerate every diff, write diagnosis and validation reports, and pass tests.	347

Table 4 Benchmark performance of DeepSeek-V4-Pro, Qwen3.5-27B Base, Qwen3.5-122B-A10B Base, and Qwen3.5-27B-RL on Terminal-Bench 2, Terminal-Bench Hard, and Long-Horizon Terminal Bench. All scores are averaged over three independent evaluation runs. The final row reports the relative performance gain of Qwen3.5-27B-RL over Qwen3.5-27B Base.

Model (Non-thinking Mode)	Terminal-Bench 2	Terminal-Bench Hard	LHTB
Deepseek-V4-Pro	51.68	36.00	30.00
Qwen3.5-27B Base	41.20	22.67	18.10
Qwen3.5-122B-A10B Base	43.82	20.00	18.85
Qwen3.5-27B-RL	49.44	32.00	22.07
Relative gain	+20.00%	+41.16%	+21.93%

Table 4 reports mean performance over three evaluation runs. Qwen3.5-27B-RL reaches 49.44% on Terminal-Bench 2, 32.00% on Terminal-Bench Hard, and 22.07% on Long-Horizon Terminal Bench. Relative to Qwen3.5-27B Base, these results correspond to improvements of 20.00%, 41.16%, and 21.93%, respectively. The largest relative gain occurs on Terminal-Bench Hard, indicating that verifier-based RL transfers to tasks constructed independently from the training pool. DeepSeek-V4-Pro remains stronger across all three benchmarks, showing that substantial performance headroom remains.

5.6 Case Study: Recursive Growth of a JSON-Diff Regression Task

To illustrate how a task evolves across rounds, we follow one lineage from the bootstrap seed to R_{15} . The original task asks the agent to run the `gendiff` command-line tool on fixed pairs of JSON fixtures and save the generated reports. Later rounds retain this objective but add configuration files, regression cases, failure diagnosis, release-note updates, and executable tests.

Table 5 shows five checkpoints in this lineage. The added difficulty comes from new executable requirements rather than longer instructions. Later versions introduce a task matrix in `tasks.json`, checks for invalid configuration, a new regression case, and consistency constraints across fixtures, expected counts, reports, and tests. By R_{15} , the agent must repair the configuration and fixture data, regenerate all reports, reconcile observed and expected counts, and pass the project test suite.

This lineage provides a concrete example of the complexity growth shown in Figures 7–9. The core objective remains unchanged, but each round adds more executable work. The agent must inspect workspace evidence, diagnose inconsistent state, repair configuration and data, regenerate artifacts, and pass executable tests. As a result, the reference solution grows from 10 to 347 non-empty shell lines without changing the underlying

task domain.

6 Conclusion

We presented RST, a framework for recursively constructing verified terminal-agent tasks. Each synthesis round extends the reference solution to introduce additional executable work, updates the verifier and public instruction to describe the same task, and validates the complete candidate in a fresh sandbox. A task is accepted only when its reference solution passes the private verifier and every tested requirement is stated in the instruction or discoverable from the workspace. Accepted tasks seed subsequent synthesis rounds and directly form the task pool for verifier-based reinforcement learning. Successful rollouts collected from the same tasks provide trajectories for supervised fine-tuning.

Across fifteen synthesis rounds, RST produces 37,484 verified tasks at a cost of approximately \$50 per 1,000 accepted tasks while maintaining stable validation yield. From R_1 to R_{15} , median solution length increases by $5.6\times$ and command use by $6.1\times$, while instruction length increases by only $1.4\times$. DeepSeek-V4-Pro pass@4 decreases from 90% to 2.5%, and mean partial credit decreases from 0.970 to 0.170, confirming that later tasks require substantially more agent capability. Supervised fine-tuning improves both Qwen3.5-27B and Qwen3.5-122B-A10B across Terminal-Bench 2, Terminal-Bench Hard, and Long-Horizon Terminal Bench. Qwen3.5-27B-RL reaches 46.07, 32.00, and 22.07 on the three benchmarks, corresponding to relative gains of 11.82%, 41.16%, and 21.93% over the base model. Domain, rewrite-family, and operator coverage remain broad across rounds, although the high-similarity tail motivates targeted deduplication during further scaling.

References

- [1] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*, 2024.
- [2] Sadia Sultana Chowa, Riasad Alvi, Subhey Sadi Rahman, Md Abdur Rahman, Mohaimenul Azam Khan Raiaan, Md Rafiqul Islam, Mukhtar Hussain, and Sami Azam. From language to action: a review of large language models as autonomous agents and tool users. *Artificial Intelligence Review*, 59(2):71, 2026.
- [3] Zhaoyang Chu, Jiarui Hu, Xingyu Jiang, Pengyu Zou, Han Li, Chao Peng, Peter O’Hearn, Earl T. Barr, Mark Harman, Federica Sarro, and He Ye. TerminalWorld: Benchmarking agents on real-world terminal tasks, 2026. URL <https://arxiv.org/abs/2605.22535>.
- [4] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026. URL <https://arxiv.org/abs/2606.19348>.
- [5] Rishi Desai, Jesse Hu, Ke Huang, Joan Cabezas, Neel Harsola, Pratyush Shukla, Daniel Wang, Xiangyi Li, Roey Ben Chaim, Adnan El Assadi, Omkaar Mukund Kamath, Fenil Faldu, Prannay Hebbar, Jiankai Sun, Yiyuan Li, Pramod Srinivasan, Ishan Gupta, Christopher Settles, Derek Chen, Pranav Raja, Albert Liu, Marek Šuppa, Nevasini Sasikumar, Luyang Kong, Erik Quintanilla, Ivan Bercovich, and Steven Dillmann. SWE-Marathon: Can Agents Autonomously Complete Ultra-Long-Horizon Software Work? <https://arxiv.org/abs/2606.07682>, 2026. Benchmark and evaluation code available at <https://github.com/abundant-ai/swe-marathon>.
- [6] Shangheng Du, Jiabao Zhao, Jinxin Shi, Zhentao Xie, Xin Jiang, Yanhong Bai, and Liang He. A survey on the optimization of large language model-based agents. *ACM Computing Surveys*, 58(9):1–37, 2026.
- [7] Tianqing Fang, Hongming Zhang, Zhisong Zhang, Kaixin Ma, Wenhao Yu, Haitao Mi, and Dong Yu. Webevolver: Enhancing web agent self-improvement with coevolving world model, 2025. URL <https://arxiv.org/abs/2504.21024>.
- [8] Shi Fu, Yingjie Wang, Yuzhu Chen, Li Shen, and Dacheng Tao. Self-Verification Provably Prevents Model Collapse in Recursive Synthetic Training. In *Advances in Neural Information Processing Systems*, 2025. URL <https://mlanthology.org/neurips/2025/fu2025neurips-selfverification/>.
- [9] Kanishk Gandhi, Shivam Garg, Noah D. Goodman, and Dimitris Papailiopoulos. Endless terminals: Scaling rl environments for terminal agents, 2026. URL <https://arxiv.org/abs/2601.16443>.

- [10] Dadi Guo, Tianyi Zhou, Dongrui Liu, Chen Qian, Qihan Ren, Shuai Shao, Zhiyuan Fan, Yi R. Fung, Kun Wang, Linfeng Zhang, and Jing Shao. Towards self-evolving benchmarks: Synthesizing agent trajectories via test-time exploration under validate-by-reproduce paradigm, 2026. URL <https://arxiv.org/abs/2510.00415>.
- [11] Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments, 2026. URL <https://doi.org/10.5281/zenodo.20953922>.
- [12] Yicheng He, Chengsong Huang, Zongxia Li, Jiaxin Huang, and Yonghui Yang. Visplay: Self-evolving vision-language models from images. *arXiv preprint arXiv:2511.15661*, 2025.
- [13] Arian Hosseini, Xingdi Yuan, Nikolay Malkin, Aaron Courville, Alessandro Sordoni, and Rishabh Agarwal. V-star: Training verifiers for self-taught reasoners. *arXiv preprint arXiv:2402.06457*, 2024.
- [14] Zhanbo Hua, Yifan Yao, Weihao Xie, Yongchi Zhao, Minghao Liu, Ruizhi Qiu, Zhewei Huang, Zun Wang, Yiyan Ji, Yunhai Ye, Letian Zhu, Xinpeng Lei, Han Li, Zhiyuan Ma, Zili Wang, Zhaoxiang Zhang, and Jiaheng Liu. Cli-universe: Towards verifiable task synthesis engine for terminal agents, 2026. URL <https://arxiv.org/abs/2606.22883>.
- [15] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-zero: Self-evolving reasoning llm from zero data, 2026. URL <https://arxiv.org/abs/2508.05004>.
- [16] Hamish Ivison, Junjie Oscar Yin, Rulin Shao, Teng Xiao, Nathan Lambert, and Hannaneh Hajishirzi. Tmax: A simple recipe for terminal agents, 2026. URL <https://arxiv.org/abs/2606.23321>.
- [17] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Learning Representations*, volume 2024, pages 54107–54157, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/edac78c3e300629acfe6cbe9ca88fb84-Paper-Conference.pdf.
- [18] Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, Ryan Bloom, Thomas Broadley, Haoxing Du, Brian Goodrich, Nikola Jurkovic, Luke Harold Miles, Seraphina Nix, Tao Lin, Chris Painter, Neev Parikh, David Rein, Lucas Jun Koba Sato, Hjalmar Wijk, Daniel M. Ziegler, Elizabeth Barnes, and Lawrence Chan. Measuring ai ability to complete long software tasks, 2026. URL <https://arxiv.org/abs/2503.14499>.
- [19] Zongxia Li, Wenhao Yu, Chengsong Huang, Zhenwen Liang, Rui Liu, Fuxiao Liu, Jingxi Che, Dian Yu, Jordan Boyd-Graber, Haitao Mi, et al. Self-rewarding vision-language model via reasoning decomposition. *arXiv preprint arXiv:2508.19652*, 2025.
- [20] Zongxia Li, Hongyang Du, Chengsong Huang, Xiyang Wu, Lantao Yu, Yicheng He, Jing Xie, Xiaomin Wu, Zhichao Liu, Jiarui Zhang, et al. Mm-zero: Self-evolving multi-model vision language models from zero data. *arXiv preprint arXiv:2603.09206*, 2026.
- [21] Zongxia Li, Zhongzhi Li, Yucheng Shi, Ruhan Wang, Junyao Yang, Zhichao Liu, Xiyang Wu, Anhao Li, Yue Yu, Ninghao Liu, Lichao Sun, Haotao Mi, and Leowei Liang. Long-horizon-terminal-bench: Testing the limits of agents on long-horizon terminal tasks with dense reward-based grading, 2026. URL <https://arxiv.org/abs/2607.08964>.
- [22] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents, 2025. URL <https://arxiv.org/abs/2308.03688>.
- [23] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang

- Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- [24] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants, 2023. URL <https://arxiv.org/abs/2311.12983>.
- [25] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym, 2025. URL <https://arxiv.org/abs/2412.21139>.
- [26] Xiaoxuan Peng, Kaiqi Zhang, Xinyu Lu, Boxi Cao, Yaojie Lu, Hongyu Lin, Xianpei Han, and Le Sun. Litecoder-terminal: Scaling long-horizon terminal environments for learning language agents, 2026. URL <https://arxiv.org/abs/2605.29559>.
- [27] Renjie Pi, Grace Lam, Mohammad Shoeybi, Pooya Jannaty, Bryan Catanzaro, and Wei Ping. On data engineering for scaling llm terminal capabilities, 2026. URL <https://arxiv.org/abs/2602.21193>.
- [28] Archiki Prasad, Weizhe Yuan, Richard Yuanzhe Pang, Jing Xu, Maryam Fazel-Zarandi, Mohit Bansal, Sainbayar Sukhbaatar, Jason Weston, and Jane Yu. Self-consistency preference optimization. *arXiv preprint arXiv:2411.04109*, 2024.
- [29] Negin Raoof, Richard Zhuang, Marianna Nezhurina, Etash Guha, Atula Tejaswi, Ryan Marten, Charlie F. Ruan, Tyler Griggs, Alexander Glenn Shaw, Hritik Bansal, E. Kelly Buchanan, Artem Gazizov, Reinhard Heckel, Chinmay Hegde, Sankalp Jajee, Daanish Khazi, Emmanouil Koukoumidis, Xiangyi Li, Hange Liu, Shlok Natarajan, Harsh Raj, Nicholas Roberts, Ethan Shen, Nishad Singhi, Michael Siu, Ashima Suvarna, Hanwen Xing, Patrick Yubeaton, Robert Zhang, Leon Liangyu Chen, Xiaokun Chen, Steven Dillmann, Saadia Gabriel, Xunyi Jiang, Anurag Kashyap, Boxuan Li, Yein Park, Minh Pham, Sujay Sanghavi, Lin Shi, Ke Sun, Yixin Wang, Zhiwei Xu, Erica Zhang, Siyan Zhao, Wanjia Zhao, Jenia Jitsev, Alex Dimakis, Benjamin Feuer, and Ludwig Schmidt. Openthoughts-agent: Data recipes for agentic models, 2026. URL <https://arxiv.org/abs/2606.24855>.
- [30] Qijia Shen, Zhiqi Huang, Vamsidhar Kamanuru, Aznaur Aliev, Jay Rainton, Ahmed Awelkair, Zhichen Zeng, Jiajun Li, Shi Dong, Yueming Yuan, Boyuan Ma, Qizheng Zhang, Jiwei Fu, Yuzhen Mao, Wendong Fan, Ping Nie, Philip Torr, Bernard Ghanem, Changran Hu, Jonathan Lingjie Li, Urmish Thakker, and Guohao Li. Seta: Scaling environments for terminal agents, 2026. URL <https://arxiv.org/abs/2607.10891>.
- [31] Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, Aaron Parisi, Abhishek Kumar, Alex Alemi, Alex Rizkowsky, Azade Nova, Ben Adlam, Bernd Bohnet, Gamaleldin Elsayed, Hanie Sedghi, Igor Mordatch, Isabelle Simpson, Izzeddin Gur, Jasper Snoek, Jeffrey Pennington, Jiri Hron, Kathleen Kenealy, Kevin Swersky, Kshiteej Mahajan, Laura Culp, Lechao Xiao, Maxwell L. Bileschi, Noah Constant, Roman Novak, Rosanne Liu, Tris Warkentin, Yundi Qian, Yamini Bansal, Ethan Dyer, Behnam Neyshabur, Jascha Sohl-Dickstein, and Noah Fiedel. Beyond human data: Scaling self-training for problem-solving with language models, 2024. URL <https://arxiv.org/abs/2312.06585>.
- [32] Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Zhicheng Dou, and Ji-Rong Wen. Envscaler: Scaling tool-interactive environments for llm agent via programmatic synthesis. *arXiv preprint arXiv:2601.05808*, 2026.
- [33] Dunwei Tu, Hongyan Hao, Hansi Yang, Yihao Chen, Yi-Kai Zhang, Zhikang Xia, Yu Yang, Yueqing Sun, Xingchen Liu, Furao Shen, Qi Gu, Hui Su, and Xunliang Cai. Scaleenv: Scaling environment synthesis from scratch for generalist interactive tool-use agent training, 2026. URL <https://arxiv.org/abs/2602.06820>.
- [34] Bertie Vidgen, Austin Mann, Abby Fennelly, John Wright Stanly, Lucas Rothman, Marco Burstein, Julien Benček, David Ostrofsky, Anirudh Ravichandran, Debnil Sur, Neel Venugopal, Alannah Hsia, Isaac Robinson, Calix Huang, Olivia Varones, Daniyal Khan, Michael Haines, Austin Bridges, Jesse Boyle, Koby Twist, Zach Richards, Chirag Mahapatra, Brendan Foody, and Osvold Nitski. Apex-agents, 2026. URL <https://arxiv.org/abs/2601.14242>.
- [35] Lilin Wang, Lucas Ramalho, Alan Celestino, Phuc Anthony Pham, Yu Liu, Umang Kumar Sinha, Andres Portillo, Onassis Osunwa, and Gabriel Maduekwe. Swe-bench++: A framework for the scalable generation of software engineering benchmarks from open-source repositories, 2025. URL <https://arxiv.org/abs/2512.17419>.
- [36] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and

- Graham Neubig. Openhands: An open platform for ai software developers as generalist agents, 2025. URL <https://arxiv.org/abs/2407.16741>.
- [37] Yiming Wang, Da Yin, Yuedong Cui, Ruichen Zheng, Zhiqian Li, Zongyu Lin, Di Wu, Xueqing Wu, Chenchen Ye, Yu Zhou, and Kai-Wei Chang. Llms as scalable, general-purpose simulators for evolving digital agent training, 2025. URL <https://arxiv.org/abs/2510.14969>.
 - [38] Yangzhen Wu, Aaron J. Li, Wenjie Ma, Li Cao, Ziheng Zhou, Mert Cemri, Shu Liu, Yuran Xiu, Chenxiao Yan, Haikun Zhao, Bin Yu, Ion Stoica, and Dawn Song. Benchevolver: Frontier task synthesis via solution-centric evolution, 2026. URL <https://arxiv.org/abs/2606.01286>.
 - [39] Yifan Wu, Yiran Peng, Yiyu Chen, Jianhao Ruan, Zijie Zhuang, Cheng Yang, Jiayi Zhang, Man Chen, Yenchu Tseng, Zhaoyang Yu, Liang Chen, Yuyao Zhai, Bang Liu, Chenglin Wu, and Yuyu Luo. Autowebworld: Synthesizing infinite verifiable web environments via finite state machines, 2026. URL <https://arxiv.org/abs/2602.14296>.
 - [40] Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning, 2025. URL <https://arxiv.org/abs/2511.16043>.
 - [41] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Oworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL <https://arxiv.org/abs/2404.07972>.
 - [42] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. Wizardlm: Empowering large pre-trained language models to follow complex instructions. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Learning Representations*, volume 2024, pages 30745–30766, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/82eec786fdfbfa53450c5feb7d1ac92-Paper-Conference.pdf.
 - [43] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
 - [44] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models, 2025. URL <https://arxiv.org/abs/2401.10020>.
 - [45] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. Multi-swe-bench: A multilingual benchmark for issue resolving, 2025. URL <https://arxiv.org/abs/2504.02605>.
 - [46] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.
 - [47] Zhiyuan Zeng, Hamish Ivison, Yiping Wang, Lifan Yuan, Shuyue Stella Li, Zhuorui Ye, Siting Li, Jacqueline He, Runlong Zhou, Tong Chen, Chenyang Zhao, Yulia Tsvetkov, Simon Shaolei Du, Natasha Jaques, Hao Peng, Pang Wei Koh, and Hannaneh Hajishirzi. RLve: Scaling up reinforcement learning for language models with adaptive verifiable environments, 2025. URL <https://arxiv.org/abs/2511.07317>.
 - [48] Ziyun Zhang, Zezhou Wang, Xiaoyi Zhang, Zongyu Guo, Jiahao Li, Bin Li, and Yan Lu. Infiniteweb: Scalable web environment synthesis for gui agent training, 2026. URL <https://arxiv.org/abs/2601.04126>.
 - [49] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data, 2025. URL <https://arxiv.org/abs/2505.03335>.
 - [50] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024. URL <https://arxiv.org/abs/2307.13854>.
 - [51] Yifei Zhou, Sergey Levine, Jason Weston, Xian Li, and Sainbayar Sukhbaatar. Self-challenging language model agents. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, pages 113959–113991.

- Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/a5a305fac88fb4ae40969cfec5eef48d-Paper-Conference.pdf.
- [52] Kaijie Zhu, Yuzhou Nie, Yijiang Li, Yiming Huang, Jialian Wu, Jiang Liu, Ximeng Sun, Zhenfei Yin, Lun Wang, Zicheng Liu, Emad Barsoum, William Yang Wang, and Wenbo Guo. Termigen: High-fidelity environment and robust trajectory synthesis for terminal agents, 2026. URL <https://arxiv.org/abs/2602.07274>.
- [53] Yiqi Zhu, Apurva Gandhi, and Graham Neubig. Training versatile coding agents in synthetic environments, 2025. URL <https://arxiv.org/abs/2512.12216>.

A Bootstrap Seed Pool Diversity

Section 4.1 describes the 639 oracle-valid bootstrap tasks, whose high-level domain distribution is shown in Figure 4. We consolidate the original fine-grained category labels into broader domains for analysis. Table 6 provides the complete mapping between the original categories and the resulting analytical groupings.

B Recursive Synthesis Implementation

The recursive synthesis system is implemented as a manifest-driven task factory in which every transformation, validation result, and round-to-round transition is recorded explicitly. Each seed-manifest entry identifies the task directory and preserves its task ID, source round, source loop, source cohort, original parent, rewrite family, rewrite operator, and repair history. Before transformation, the factory verifies that the seed contains the required task components, including the public instruction, task configuration, oracle solution, verifier entry point, verifier checks, and environment definition. The complete seed directory is then copied into an isolated output directory. Generation is restricted to six task files: `instruction.md`, `solution/solve.sh`, `tests/test.sh`, `tests/test_state.py`, `environment/Dockerfile`, and `task.toml`. File writes outside this allowlist or outside the copied task directory are rejected.

Rewrite targets are selected using a taxonomy of 40 operators grouped into five families: environment and runtime substrate, build and execution, data and artifact processing, configuration and state migration, and diagnostics and auditing. For each seed, a local affordance scan examines bounded representations of its instruction, solution, verifier, configuration, and environment files. It scores operators according to observable signals such as package manifests, structured-data formats, build scripts, command-line interfaces, archives, databases, logs, permissions, services, and version constraints. Operator selection combines this local compatibility score with a family-balance term and an inverse-frequency penalty for operators already assigned within the current batch. This prevents a small number of broadly applicable operators from dominating the generated pool. The selector records one preferred operator, up to five feasible alternatives, and operators that should not be applied to the seed. An additional model-based ranking step evaluates whether the preferred operator is natural, safe, and sufficiently supported by the seed. It may retain the preferred operator or replace it with one of the recorded alternatives, but it cannot introduce an operator that was excluded by the local scan.

The selected operator is first converted into an explicit transformation contract rather than being applied directly. This contract specifies the original behavior that must be preserved, the new terminal-native requirement, the expected sequence of inspection, derivation, execution, validation, and finalization, and the observable artifacts associated with each stage. It also defines discoverable evidence, intermediate outputs, final outputs, verifier checks, and conditions that reject empty, stale, hard-coded, or verifier-specific shortcuts. The contract requires at least four distinguishable checks covering evidence discovery, intermediate-state validity, final semantic correctness, and shortcut rejection. It separately records what must appear in the public instruction and what may remain implicit because it can be discovered from local documentation, configuration files, fixtures, logs, source files, or normal command output.

Task construction then proceeds through ordered, file-scoped stages. The oracle-solution stage extends `solution/solve.sh` so that it completes both the preserved seed behavior and the new requirement from the initial sandbox state. The verifier stage derives checks from the transformation contract rather than copying incidental commands from the solution. It checks semantic content and state, preserves relevant checks from the parent task, and rejects solutions that produce only placeholder artifacts. The instruction stage expresses the user-visible goal, a small number of fair starting points, and the required deliverable without exposing private verifier paths, exact solution commands, or a complete acceptance checklist. An environment-alignment stage is invoked only when the extension requires a local dependency, fixture, configuration, or metadata change. After these stages, a cross-file consistency pass compares the instruction, solution, verifier, environment, and transformation contract. It repairs discrepancies such as artifacts produced by the solution but omitted from the contract, verifier requirements unsupported by public evidence, or instruction requirements that are not

Table 6 Mapping from the high-level domain labels in Figure 4 to the original categories for the 639 oracle-valid bootstrap tasks. Domains are post-hoc merges of overlapping raw categories; they are not native dataset fields.

Domain	#Tasks	Original TerminalWorld categories
Build & Compile	149	build-system, software-development, software-engineering
Scripting & Automation	103	scripting, scripting-automation
System Admin & Services	87	process-management, system-administration
Security / Reverse / Crypto	52	security
Package & Environment	47	environment-setup, package-management
Version Control & Release	34	version-control
Containers & Kubernetes	34	container-cloud-native, container-operations, container-orchestration
Database & Storage	21	data-management, database, database-operations
Files, Documents & Media	21	document-processing, documentation, file-management, file-operations, text-processing
Network / Web / API	20	blockchain, networking
Debug, Test & Quality	12	debugging, debugging-troubleshooting, performance-optimization, testing
Cloud, Deploy & DevOps	12	cloud-infrastructure, cloud-services, deployment, devops, distributed-computing, monitoring-observability
Scientific / HPC / Bio	11	bioinformatics, scientific-computing
Other / Unknown	10	unknown
Education / Games / Misc	8	coursework-exercises, games-entertainment, program-execution
Data Processing & Analysis	8	data-analysis, data-processing
Language Tooling	7	developer-tools, development-tools, programming, system-programming, systems-programming
ML / AI	2	ml-training
Virtualization	1	virtualization

evaluated.

Two deterministic gates are applied before sandbox execution. The generation-quality gate verifies that the candidate represents a substantive cross-file transformation. In the current implementation, at least three task files must change, the oracle solution must contain at least eight changed lines, and the verifier must contain at least twelve changed lines. The same gate rejects instructions that expose private test paths, resemble step-by-step command recipes, enumerate excessive implementation details, exceed 180 words, or expand to more than 1.6 times the length of the parent instruction. The static-preflight gate checks Docker build consistency, required task files, critical artifact paths, and the correspondence between artifacts required by the verifier and artifacts produced by the solution or environment. It also verifies that evidence declared discoverable in the contract is present in the task bundle. Failures are recorded separately as contract, generation-quality, or static-preflight failures, which allows these losses to be distinguished from failures during executable validation.

Candidates that pass the deterministic gates are evaluated with the oracle agent in fresh Daytona sandboxes through Harbor. Each candidate receives one oracle execution, after which the private verifier is run in the resulting environment. A candidate is oracle-passed only when the verifier returns a reward of one and the trial contains no execution exception. Other outcomes are classified as zero reward, build failure, timeout, runtime error, excessive output, or unknown failure. Failed candidates from configured repair categories enter a bounded feedback-repair procedure. The repair model receives the transformation contract together with truncated build logs, execution logs, verifier results, and failure metadata. It must identify a specific failure type and may modify only the six allowed task files. Every repaired candidate is subjected to static preflight again and is then revalidated in a new sandbox. The reported configuration permits at most two feedback-repair rounds, preventing repeated unconstrained regeneration from obscuring the original transformation. The final manifest includes only candidates that pass the last validation attempt and records their initial result, final result, rewrite metadata, repair count, task directory, parent identifier, and provenance.

Round-to-round orchestration begins by reading the `passed_tasks` records from every `final_manifest.json` produced by the preceding round. Records are deduplicated by resolved task path while retaining the source run, loop, cohort, family, operator, repair count, and original parent. The resulting `pool_manifest.jsonl` is the complete candidate pool for the next round. Seed selection assigns each candidate a score based on static risk, environment cost, rewrite metadata, and validation history. Long build timeouts and resource-intensive environment features receive penalties. Tasks requiring one repair remain eligible because they have subsequently passed executable validation, while candidates requiring repeated repair receive a stability penalty. When rollout statistics are available, the selector favors tasks with nontrivial but nonzero success rates, low infrastructure-error rates, and intermediate interaction lengths. Tasks with very high pass rates are penalized as potentially too easy, while tasks with almost no successful executions or high error rates are penalized as potentially unstable.

After scoring, candidates are sorted deterministically and selected subject to diversity constraints over original parent, task category, rewrite family, and source cohort. The standard configuration targets 1,000 seeds and allows at most four descendants from one parent, 160 tasks from one category, 320 tasks from one rewrite family, and 280 tasks from one source cohort. If these constraints prevent the target size from being reached, the controller relaxes them according to a predetermined schedule and records the cap configuration used for each attempt. It does not silently fill the remaining positions without diversity constraints. The selected task bundles are then materialized into a local seed directory, and a new manifest is written with both the materialized path and the original source path. Materialization verifies that every selected bundle can be copied successfully and that its task configuration is present before synthesis is launched.

The selector and materializer additionally protect recursive ancestry. Parent identifiers are recovered from the explicit `parent_seed`, `original_seed`, `source_task_id`, task ID, and source path fields. Because recursively generated task names may exceed filesystem limits, long names are compacted to at most 96 characters while preserving the original bootstrap-parent identifier and appending a stable hash. For manifests containing at least 20 records, the controller requires at least 80% of records to retain recoverable bootstrap lineage. The same check is applied before selection, after selection, and after materialization. A failed check terminates

the round before generation. This safeguard is necessary because loss of parent identity would make the per-parent cap ineffective and would invalidate lineage-coverage and parent-child novelty analyses.

C Operator Taxonomy

The synthesis factory organizes rewrite targets according to five sources of executable difficulty: the runtime environment in which commands operate, the build and execution process, the artifacts produced or transformed, the configuration or persistent state being maintained, and the diagnostic evidence required to identify and resolve failures. We choose these five families because they capture the five distinct locations at which additional difficulty can be introduced into a terminal task. *Environment and runtime substrate* covers the execution conditions that must be established before a task can run, such as dependencies, permissions, paths, processes, and resource constraints. *Build, test, and execution* covers the command sequence and program behavior required to complete the task. *Data, artifact, and report processing* covers transformations whose primary objective is to produce or validate concrete outputs. *Configuration and state migration* covers persistent or cross-file state that must remain consistent across versions, executions, or recovery operations. *Diagnostics, audit, and forensics* covers tasks in which the central challenge is to infer the required action from logs, traces, failures, permissions, or other system evidence. This division was selected according to three criteria. First, it covers the principal sources of complexity encountered in terminal interaction, from execution setup to evidence-based diagnosis. Second, every family can be instantiated as observable file, process, command, or system-state changes, making generated tasks executable and automatically verifiable. Third, the families describe transformation mechanisms rather than application domains such as software engineering, data processing, or security, whose boundaries overlap and do not specify how a seed should be made harder. Finer distinctions are represented by eight operators within each family, while assigning each rewrite to its dominant mechanism keeps the round-level distribution measurable and prevents an excessively fragmented taxonomy. Each operator is therefore accompanied by an executable card specifying compatible seed affordances, construction patterns, evidence sources, expected artifacts, verifier and instruction strategies, and shortcut-rejection criteria. Table 7 lists the complete taxonomy used by the local affordance scan and contract-construction stage.

D Local Filters, Preflight, and Repair Policy

Generation quality filter. Before sandbox execution, the pipeline checks whether a rewrite is both substantive and publicly solvable. A candidate is rejected if it uses an unknown operator, changes fewer than three tracked files, modifies the solution by fewer than eight lines, or modifies the verifier by fewer than twelve lines. These thresholds prevent superficial or verifier-only rewrites and are not used as difficulty measures. The filter also rejects instructions that expose private test paths, prescribe test-driven acceptance loops, exceed 180 words or $1.6\times$ the seed length, enumerate excessive paths or commands, or reveal verifier details. Borderline cases are retained with warnings.

Static preflight. Static preflight checks cross-file executability before costly sandbox validation. It detects malformed Dockerfiles, missing COPY sources, and other environment inconsistencies. It also verifies that artifacts required by the contract or verifier are produced by the solution or provided by the environment, and that promised evidence is discoverable from the task files. Missing shell safety conventions, such as a shebang or `set -e`, produce warnings rather than rejection. Candidates with hard preflight failures are not submitted to Daytona.

Sandbox oracle validation and feedback repair. Candidates passing both static gates are executed by the reference solution in a fresh Daytona sandbox and accepted only if the verifier returns full reward without an execution exception. Failures are classified as zero reward, build failure, runtime error, timeout, or unknown failure. Recoverable failures may enter a bounded repair loop conditioned on the task contract and Daytona logs. Repair may modify only the solution, verifier, instruction, environment, and task configuration files. It

may correct implementation or contract inconsistencies, but cannot weaken semantic checks, remove shortcut protection, or expose private requirements in the instruction. Each repaired task must pass static preflight and Daytona validation again. Candidates that still fail after the configured repair rounds are excluded from the accepted manifest.

D.1 Task Quality Control and Validation

Requirement discoverability. The pipeline identifies every condition that can change the verifier reward and checks whether the agent can know that condition before acting. Each condition is handled in one of four ways:

- A semantic requirement with no public source must be stated directly in the instruction.
- A requirement documented in a local README, specification, configuration, or fixture may remain there, but the instruction must provide a clear starting point for finding it.
- A secondary deliverable must be named in the instruction, while its detailed schema may remain in discoverable workspace documentation.
- A check that only rejects placeholders, stale outputs, or hard-coded answers need not be stated because it adds no new semantic requirement.

Thus, the verifier cannot require behavior that is neither stated nor discoverable.

Public-instruction boundary. The public instruction specifies the task objective and every reward-relevant condition that cannot be discovered from the workspace. It does not expose verifier implementation details, private test paths, or the commands used by the oracle solution. When an exact value, path, or schema is already available in a local README, specification, configuration, or fixture, the instruction identifies that evidence source instead of repeating its contents. This keeps the task concise without hiding information required for success.

Contract-granularity variants. To study the effect of instruction specificity, an optional analysis stage creates five instruction-verifier variants while keeping the environment and oracle solution fixed. The variants range from a goal-level exploratory contract to a nearly complete statement of the acceptance conditions. At lower granularities, a condition omitted from the instruction must also be removed from the verifier. At higher granularities, retained verifier conditions must be stated explicitly or linked to discoverable workspace evidence. Consequently, no variant evaluates an undisclosed semantic requirement.

E Synthesis Prompt Templates

This appendix presents the core prompt templates used by RST. All fixed instructions are reproduced verbatim from the implementation. Run-dependent inputs, including seed-task files, operator cards, task contracts, and Daytona feedback, are replaced with angle-bracket placeholders labeled by the functions that supply them at execution time.

Table 7 Rewrite operator taxonomy used by RST. Family IDs match the implementation; operator IDs are the soft targets assigned before staged rewrite.

Family	Operator ID	Definition
environment runtime substrate	dependency_version_- alignment	Align dependency/runtime versions and verify the toolchain.
	path_workdir_alignment	Resolve working-directory and path/entrypoint mismatches.
	permission_executable_- alignment	Repair executable bits, ownership, or access needed by the workflow.
	environment_variable_- resolution	Infer/configure env vars, locale, timezone, shell init, or profiles.
	toolchain_availability_- check	Ensure required compilers/interpreters/CLI tools exist.
	container_build_alignment	Align Dockerfile, build context, and image setup.
	service_process_lifecycle	Start/stop/inspect local services, daemons, or ports.
build/test execution workflow	resource_cleanup_and_limits	Handle temp files, timeouts, log growth, and cleanup.
	compile_link_package_- workflow	Compile, link, package, or assemble a runnable artifact.
	unit_test_failure_repair	Run unit tests, interpret failures, and complete the workflow.
	integration_test_workflow	Coordinate multiple components and validate integrated behavior.
	cli_argument_behavior	Validate CLI flags, stdio, exit codes, and argument behavior.
	lint_format_static_check	Run lint/format/type-check and resolve surfaced issues.
	runtime_error_debug_loop	Diagnose runtime exceptions/stack traces and repair behavior.
data/artifact report processing	build_artifact_generation	Produce a binary/package/bundle and verify usability.
	performance_or_benchmark_- smoke	Run a bounded benchmark/smoke check and record a result.
	format_conversion_- validation	Convert formats while validating semantic preservation.
	schema_content_validation	Validate schema, fields, types, ranges, or completeness.
	aggregation_summary_report	Compute aggregates and produce a summary/report.
	artifact_inventory_- reconciliation	Reconcile outputs, manifests, and directory inventories.
	archive_compression_- extraction	Extract/pack/inspect archives such as tar/zip/gzip.
configuration state migration	dedup_sort_normalization	Deduplicate, sort, normalize, or canonicalize artifacts.
	checksum_hash_provenance	Generate or verify hashes/checksums/provenance.
	media_or_binary_metadata_- processing	Inspect lightweight media/binary metadata.
	config_data_consistency	Reconcile configuration and data/source consistency.
	manifest_lockfile_- reconciliation	Align manifests, lockfiles, or package metadata.
	state_migration_transform	Migrate state across formats/versions/layouts.
	cache_index_regeneration	Regenerate caches, indexes, or derived metadata.
diagnostics audit forensics	database_or_file_state_- initialization	Initialize and validate file-backed/DB state.
	template_render_consistency	Keep templates and rendered outputs consistent.
	profile_feature_flag_- selection	Select/validate profiles or feature-flag branches.
	backup_rollback_idempotency	Support backup, rollback, and idempotent reruns.
	log_error_diagnosis	Inspect logs/stderr, identify cause, complete workflow.
	trace_event_correlation	Correlate logs/events/timestamps across sources.
	verifier_failure_- interpretation	Use local validation output to infer missing state.
	security_permission_audit	Audit permissions, sensitive files, or unsafe config.
	data_quality_anomaly_- investigation	Investigate missing/duplicate/anomalous data.
	process_endpoint_inspection	Inspect local processes, ports, sockets, or endpoints.
	git_history_diff_forensics	Use git history/diff to diagnose or produce state.
	evidence_bundle_generation	Create a verifiable evidence bundle with logs/reports.

SYSTEM Prompt

You generate high-quality TerminalWorld task transformations.

Return ONLY valid JSON. Do not use markdown.

Rules:

- Keep the original task recognizable.
- Rewrite only files explicitly requested in this step.
- Return full file contents, not patches.
- Do not add benchmark canary text.
- Do not add network-only requirements or heavyweight new dependencies unless the seed already naturally uses them.
- The final task must be solvable by the rewritten solution and verifiable by tests.
- The user instruction must not leak the full solution or private test harness paths.
- The rewritten task should preserve the seed's main workflow and add one realistic terminal-native subgoal.
- Prefer tasks that require filesystem inspection, CLI composition, config/data validation, log/error interpretation, and observable artifact checks.
- Do not turn the task into a narrow "fix this bug" prompt; any repair/debugging should be one part of a larger goal-oriented workflow.
- Keep instruction.md a compact public goal, not an acceptance rubric. Only publish undiscoverable hard requirements; put discoverable details in workspace docs and point to them. Never leave acceptance criteria only inside tests/, and never dump path inventories, field schemas, or step checklists that invite reward hacking.

AFFORDANCE RANKING Prompt

AFFORDANCE RANKING.

Role:

Evaluate whether the proposed rewrite target is natural for this seed task. This is a ranking step only; do not rewrite files.

Inputs:

- A preferred operator selected by the pipeline using local file signals and diversity quotas.
- Several fallback operators from the same local feasibility scan.
- The current seed task files.

Decision rules:

- Mark the preferred operator high only if it can be supported by existing seed affordances or small local evidence additions.
- If the preferred operator would be artificial, rank fallback operators that better fit the seed.
- Do not invent a new taxonomy outside the provided operators.
- Prefer operators that require terminal exploration and verifiable artifacts, not superficial wording changes.
- Judge feasibility against the operator card's seed_affordances, construction_pattern, expected_artifacts, and anti_shortcut_strategy.

Return schema:

```
{
  "status": "ok|blocked",
  "ranked_operators": [
    {"operator": "operator_id", "fit": "high|medium|low", "reason": "short reason"}
  ],
  "blocked_operators": [
    {"operator": "operator_id", "reason": "why it is artificial or unsafe for this seed"}
  ],
  "rationale": "short summary"
}
```

Rewrite target context:

```
<json.dumps(target_context, ensure_ascii=False, indent=2)>
```

Seed task:

```
<json_task_context(task_dir)>
```

STEP 0: Task Transformation Contract Prompt (Part 1/3)

STEP 0: task transformation contract. Do not rewrite files in this step.

Role:

Design one controlled, verifiable TerminalWorld rewrite contract. The task must preserve the seed's core workflow and add one terminal-native requirement that increases workflow depth.

Rewrite target protocol:

- Use the preferred operator if it is natural for this seed.
- If the preferred operator is artificial or unsafe, choose one fallback operator and explain why.
- Do not choose blocked operators.
- Do not collapse to config/data consistency unless the seed genuinely exposes config/data relationships.
- The operator should determine the main transformation mechanism, but the task must still feel like a normal terminal workflow.

Operator card protocol:

- Treat the selected operator card as a simple construction recipe, not a label.
- Use the card's `construction_pattern` to build a concrete `task_chain`.
- Use the card's `evidence_sources` to decide where hidden details should be discoverable.
- Use the card's `expected_artifacts` to define evidence, intermediate, and final artifacts.
- Use the card's `verifier_strategy` and `anti_shortcut_strategy` to define `reward_checks`.
- Use the card's `instruction_strategy` as a style hint for tone/compactness, not as a license to hide acceptance criteria.

Quality criteria:

- The task starts from the original seed initial state, not from a halfway failure state.
- The new requirement is user-visible through artifacts, command output, state, or reports.
- Details may be hidden from instruction only when they remain discoverable through local files, scripts, configs, fixtures, logs, or normal command output that the instruction fairly points to.
- The required work should involve inspection, derivation, execution, validation, and finalization, not a single deterministic file write.
- The verifier can check semantic correctness and reject shortcuts, but must not invent silent acceptance criteria.
- The contract must define a short `task_chain` and at least four observable `reward_checks` for dense RL-style feedback.
- The public instruction must remain solvable without verifier access, but stay compact to avoid reward hacking: mention the goal, fair start, main deliverable/completion signal, and only those exact constants that are not discoverable from workspace docs.
- Prefer `instruction_boundary.hide` + workspace evidence over inlining schemas, path inventories, or operational steps. Never hide a semantic check that exists only in the verifier with no discoverable evidence.

Hard constraints:

- superficial refactors of solution style;
- merely strengthening tests without changing the user-visible task;
- hidden-only requirements that instruction never implies and the agent cannot discover from workspace evidence;
- adding large dependencies, internet access, services, or long-running workloads;
- creating a task that is just "fix a bug";
- telling the agent to inspect or rerun `/tests`, `tests/test.sh`, or `pytest` as the acceptance loop.

STEP 0: Task Transformation Contract Prompt (Part 2/3)

```
Return schema:
{
  "status": "ok|blocked",
  "rewrite_family": "selected family id",
  "rewrite_operator": "selected operator id",
  "operator_fit": "preferred|fallback",
  "why_fit": "short reason why this operator naturally fits the seed",
  "goal": "one concise user-facing goal for the rewritten task",
  "preserved_workflow": "what original workflow remains",
  "new_requirement": "the added terminal-native requirement",
  "task_chain": [
    {"stage": "inspect", "artifact_or_state": "local config/schema/log/source/build output to inspect"},
    {"stage": "derive", "artifact_or_state": "rule/schema/path/expected state to infer from local evidence"},
    {"stage": "execute", "artifact_or_state": "intermediate or final artifact/state to create by running or editing"},
    {"stage": "validate", "artifact_or_state": "local validation/check result or parseable evidence"},
    {"stage": "finalize", "artifact_or_state": "final deliverable or checked state"}
  ],
  "expected_artifacts": [
    {"path": "relative or absolute path used by the task", "role": "evidence|intermediate|final", "requirement": "observable content/state requirement"}
  ],
  "reward_checks": [
    {"name": "check_01_required_evidence", "checks": "discoverable evidence source exists and is used or referenced"},
    {"name": "check_02_intermediate_artifact", "checks": "intermediate artifact/state is present, parseable, and non-empty"},
    {"name": "check_03_final_semantics", "checks": "final output/state satisfies the semantic task goal"},
    {"name": "check_04_no_shortcut", "checks": "reject placeholder, empty, stale, hardcoded, or verifier-only outputs"}
  ],
}
```

STEP 0: Task Transformation Contract Prompt (Part 3/3)

```
"instruction_boundary": {
  "mention": ["public goal", "at most 1-2 fair starting points", "main final deliverable or completion signal", "only undiscoverable hard acceptance constants"],
  "hide": ["step-by-step commands", "path/field inventories", "schema/format dumps", "oracle repair recipe", "anti-shortcut implementation details", "private harness paths under tests/", "discoverable details that belong in local docs"]
},
"environment_changes": ["none, or small local evidence/dependency changes needed"],
"rationale": "why the selected operator fits and why the rewrite is harder but controllable"
}

Rewrite target context:
<json.dumps(target_context, ensure_ascii=False, indent=2)>

Current seed task:
<json_task_context(task_dir)>
```

STEP 1: Oracle Solution Prompt

STEP 1: oracle solution protocol. Rewrite solution/solve.sh only.

Role:

Write the oracle shell implementation for the contract. The solution must complete the full workflow from the original initial state and behave like a strong terminal agent's successful trajectory.

Implementation requirements:

- Preserve the seed's main workflow and add the selected rewrite operator's requirement.
- Implement every stage in `contract.task_chain` in order: inspect, derive, execute, validate, finalize.
- Inspect inputs before transforming them; do not blindly overwrite final artifacts.
- Create or update every expected artifact with semantically meaningful content.
- Create observable intermediate/final artifacts needed by `contract.reward_checks`.
- Include bounded validation of intermediate artifacts before writing final outputs.
- Keep the script deterministic, idempotent where reasonable, and safe under repeated execution.
- Use local files, CLI tools, configs, fixtures, logs, or build/test commands that a terminal agent could discover.

Forbidden shortcuts:

- Hardcoding verifier-only constants without deriving or justifying them from discoverable evidence.
- Replacing the task with a no-op or a single trivial echo/write unless the seed itself demands that.
- Removing seed behavior to make the new verifier easier.
- Depending on internet access, external services, or unbounded background work.

Return schema:

```
{
  "status": "ok|blocked",
  "rationale": "short reason",
  "files": {"solution/solve.sh": "FULL FILE CONTENT"}
}
```

Task contract:

```
<json_contract_context(contract)>
```

Current task:

```
<json_task_context(task_dir)>
```

STEP 2: Verifier Prompt

STEP 2: verifier protocol. Allowed outputs are tests/test_state.py and tests/test.sh.

Role:

Write the oracle verifier for the contract, not for incidental implementation details in solution/solve.sh.

Verification requirements:

- Check the user-visible goal, preserved seed behavior, and all expected artifacts.
- Turn contract.reward_checks into distinct verifier subchecks where possible, using clear names that match the reward check names.
- Include at least four dense checks: evidence/discovery, intermediate artifact validity, final semantic correctness, and no-shortcut rejection.
- Verify semantic content/state, not only file existence.
- Enforce no-shortcut checks from contract.reward_checks.
- Preserve important seed-task checks unless the contract explicitly supersedes them.
- Prefer robust predicates over brittle command-order or exact implementation checks.
- Keep tests deterministic, fast, and compatible with Daytona sandbox execution.

Failure quality:

- Error messages should help diagnose missing artifacts, wrong formats, or inconsistent state.
- Do not leak a full solution recipe through assertion messages.

Return schema:

```
{
  "status": "ok|blocked",
  "rationale": "short reason",
  "files": {
    "tests/test_state.py": "FULL FILE CONTENT",
    "tests/test.sh": "FULL FILE CONTENT"
  }
}
```

Task contract:

<json_contract_context(contract)>

Current task:

<json_task_context(task_dir)>

STEP 3: Public Instruction Prompt (Part 1/2)

STEP 3: public instruction rewrite (discoverability-filtered). Rewrite instruction.md only.

Role:

Write instruction.md as a fair public user request for a terminal agent in Terminal-Bench / TerminalWorld style. Solution already exists. Do NOT treat the verifier as a rubric to copy.

Example of desired style (do NOT copy content; match brevity):

Set up a Python project using Poetry in /app. Create a '.gitignore' containing 'test/inner_project/inner_project', run 'poetry install', verify 'inner_project' imports in that environment, then use the project's console script to write a valid JSON report at '/app/report.json' (see '--help' for arguments).

Critical anti-hacking rule:

- Dumping many absolute paths, schema fields, exact formats, or numbered operational steps into instruction.md makes agents shortcut / reward-hack.
- Prefer a compact goal + pointers to local docs. Put discoverable detail in workspace files, not in the instruction.

Path budget (match Terminal-Bench / TerminalWorld norms):

- Absolute paths are allowed, but usually only 1-2 (workspace entry like /app plus one main deliverable).
- At most ~3 absolute paths total. Never list path inventories of intermediate/meta artifacts.
- If more locations matter, point to a local README/spec/config instead of listing them.

Inputs you may use:

- current instruction
- oracle solution (workflow context only)
- a discoverability-filtered acceptance checklist
- the task contract boundary

Do NOT use or restate the full tests/ contents. The checklist already filtered them.

Fairness / discoverability (same policy as instruction repair):

- MUST include items under must_publish_in_instruction (missing from instruction AND not discoverable in workspace docs)
 - . Prefer the primary deliverable path over secondary ones if the list is long.
- For point_to_workspace_docs_instead: tell the agent to follow those files; do NOT restate their full contents.
- meta_artifacts_name_only: mention deliverable names if needed; no internal schemas.
- dropped_as_discoverable: do NOT dump these into the instruction.

STEP 3: Public Instruction Prompt (Part 2/2)

Style / restraint:

- Direct imperative, 1-3 short paragraphs. Prefer prose over bullets.
- No first-person chat ("I need you", "We need", ...).
- Keep close to current instruction length; at most ~1.4x current word count.
- Target about 80-130 words unless must-publish items require a little more.
- Do NOT expand field-level schemas for manifests/checksums/evidence.
- Do NOT enumerate every meta file's internal format.
- Do NOT paste solve.sh commands, anti-shortcut lines, or reward-check names.
- Do NOT write numbered walkthroughs / step checklists.

Hard bans:

- any path under /tests or reference to tests/test.sh, tests/test_state.py, local_check.sh under /tests;
- pytest / "run the tests" / "until tests pass" / "rerun until" / "the test suite fully specifies" as the acceptance loop;
- never tell the agent to open private harness files for acceptance criteria.

Use of the contract:

- Follow instruction_boundary.mention for public essentials.
- Keep instruction_boundary.hide out of the instruction.

Return schema:

```
{
  "status": "ok|blocked",
  "rationale": "short reason (fairness without rubric dump)",
  "files": {"instruction.md": "FULL FILE CONTENT"}
}
```

Task contract:

```
<json_contract_context(contract)>
```

Current instruction.md:

```
““
<instr>
““
```

solution/solve.sh (context only):

```
““
<solve>
““
```

Acceptance checklist (discoverability-filtered; NOT the full verifier):

```
<json.dumps(checklist, ensure_ascii=False, indent=2)>
```

STEP 4: Environment Alignment Prompt

STEP 4: environment alignment protocol.

Use the task contract as the source of truth.

Rewrite environment/Dockerfile and/or task.toml only if the contract requires a small environment metadata or dependency alignment.

Default preference: return an empty files object.

If you change environment files:

- preserve the seed's base image and installation style;
- keep the build context self-contained and ensure every COPY source exists;
- do not add internet-only runtime behavior;
- do not add proxies, secrets, credentials, or external service assumptions;
- keep storage, memory, and time requirements within normal Daytona limits;
- make only the minimum dependency or metadata change needed by the rewritten task.
- if the instruction intentionally hides exact constants, formats, paths, or checks, ensure those details are discoverable through existing local files or Dockerfile-created local files/scripts/fixtures.
- do not rely on the verifier as the only place where hidden details exist.
- do not introduce unguarded sed/perl edits against externally cloned dependency paths; prefer local fixtures or guard target files with test -f before editing.
- do not inflate resource requests or task timeout unless the existing seed already requires it.

Return schema:

```
{
  "status": "ok|blocked",
  "rationale": "short reason",
  "files": {
    "environment/Dockerfile": "FULL FILE CONTENT IF CHANGED",
    "task.toml": "FULL FILE CONTENT IF CHANGED"
  }
}
```

Task contract:

```
<json_contract_context(contract)>
```

Current task:

```
<json_task_context(task_dir)>
```

STEP 5: Consistency Repair Prompt

STEP 5: consistency repair protocol.

Fix cross-file inconsistencies so the oracle solution should pass the verifier in a fresh sandbox build.

Mandatory audit procedure:

1. List every file/path the verifier tests require to exist or contain specific content.
2. List every file/path solution/solve.sh creates or updates.
3. List every file/path promised in contract.expected_artifacts and every artifact/state implied by contract.task_chain.
4. List every dense reward check in contract.reward_checks and identify which verifier subcheck enforces it.
5. Compare these lists; any verifier-required path or reward-check artifact missing from the solution plan is a bug.
6. Mentally execute solution/solve.sh in order; if a step would fail or skip an artifact, fix the solution.
7. If environment/Dockerfile is supposed to provide config/scripts/fixtures, confirm they are actually created in the Dockerfile.
8. If instruction points to a path, config, validation script, README, fixture, or log, confirm that evidence exists or is created by the environment.
9. Instruction fairness audit:
 - If instruction mentions /tests, tests/test.sh, pytest acceptance loops, or "until tests pass", remove those leaks.
 - Prefer fixing unfair gaps by adding/repairing workspace evidence (README/spec/config) that instruction already points to - not by dumping verifier paths/fields/steps into instruction.md.
 - Only publish an exact string/path/completion constant in instruction.md when it is undiscoverable from workspace evidence.
 - Do not weaken verifier checks to hide unfair gaps.
 - Do not turn instruction into a verifier dump, schema inventory, or numbered walkthrough (reward hacking).

Repair only genuine consistency problems. Do not change the selected rewrite_family/rewrite_operator or invent a new task.

Prefer fixing solution/environment/instruction mismatches over weakening verifier checks.

Keep instruction.md compact; expand only for a few undiscoverable hard constants.

Return full file contents only for changed files. Do not create new files outside this whitelist:

solution/solve.sh, tests/test_state.py, tests/test.sh, instruction.md, environment/Dockerfile, task.toml.

Return schema:

```
{
  "status": "ok|blocked",
  "rationale": "short reason",
  "verifier_required_paths": ["paths or artifacts checked by verifier"],
  "solution_created_paths": ["paths or artifacts created by solution"],
  "contract_promised_paths": ["paths or artifacts promised by contract.expected_artifacts/task_chain/reward_checks"],
  "issues_found": ["specific cross-file inconsistency, or empty list"],
  "repairs_made": ["specific repair made, or empty list"],
  "files": {
    "solution/solve.sh": "FULL FILE CONTENT IF CHANGED",
    "tests/test_state.py": "FULL FILE CONTENT IF CHANGED",
    "tests/test.sh": "FULL FILE CONTENT IF CHANGED",
    "instruction.md": "FULL FILE CONTENT IF CHANGED",
    "environment/Dockerfile": "FULL FILE CONTENT IF CHANGED",
    "task.toml": "FULL FILE CONTENT IF CHANGED"
  }
}
```

Task contract:

<json_contract_context(contract)>

Current task:

<json_task_context(task_dir)>

F Training and Evaluation Implementation

The SFT launchers separate cluster orchestration from experiment configuration. A multi-node wrapper initializes Ray over the cluster network and submits SLIME training as a Ray job. The 64-GPU Qwen3.5-72B configuration uses eight nodes with eight GPUs each, terminal-agent trajectories stored as message and metadata records, Qwen3.5-specific loss masking, TP4/PP2/CP2 parallelism, Adam with cosine decay, optimizer CPU offload, and flash attention. The YAML-based launcher provides the same training path for four-node runs while exposing the data, checkpoint, schedule, and optimizer settings as configuration fields. The combined-data experiment trains for one epoch on 10,778 examples with a global batch size of 128, a maximum context length of 262,145, and learning rates of 3×10^{-6} to 3×10^{-7} . Qwen3.5-122B-A10B reuses the same orchestration layer with a model-specific script and TP2/PP8/CP2/EP4 parallelism for its

DAYTONA FEEDBACK REPAIR Prompt

DAYTONA FEEDBACK REPAIR.

The task has already been generated and then run in Daytona oracle validation.

Use the dynamic failure logs below as the source of truth. Repair only the files needed so solution/solve.sh passes the verifier in Daytona.

Rules:

- If result.json shows build failure, repair environment/Dockerfile or task.toml.
- If verifier says an artifact is missing, make solution/solve.sh create it, unless the artifact should be provided by environment.
- If verifier says content is malformed/empty, fix solution generation logic.
- If verifier and contract disagree, minimally repair the verifier to match the contract.
- Do not weaken verifier just to pass; preserve anti-shortcut checks.
- Keep instruction compact. Do not copy Daytona verifier failures, hidden expected values, complete artifact lists, schema fields, reward checks, or exact assertions into instruction.md.
- Do not expand instruction.md during repair unless it is inconsistent or impossible to start from; if editing it is necessary, keep it as a public request with at most two concrete entry points.
- Return full file contents only for changed files.
- Do not create new files. Do not return any path outside this whitelist:
solution/solve.sh, tests/test_state.py, tests/test.sh, instruction.md, environment/Dockerfile, task.toml.

Return schema:

```
{
  "status": "ok|blocked",
  "failure_type": "build_failed|solution_artifact_missing|solution_content_wrong|verifier_mismatch|
    environment_missing_evidence|other",
  "root_cause": "specific reason based on Daytona logs",
  "repairs_made": ["specific file-level repairs"],
  "files": {
    "solution/solve.sh": "FULL FILE CONTENT IF CHANGED",
    "tests/test_state.py": "FULL FILE CONTENT IF CHANGED",
    "tests/test.sh": "FULL FILE CONTENT IF CHANGED",
    "instruction.md": "FULL FILE CONTENT IF CHANGED",
    "environment/Dockerfile": "FULL FILE CONTENT IF CHANGED",
    "task.toml": "FULL FILE CONTENT IF CHANGED"
  }
}
```

Task contract:

```
<json_contract_context(contract)>
```

Current task files:

```
<json_task_context(task_dir)>
```

Daytona failure evidence:

```
<json.dumps(evidence, ensure_ascii=False, indent=2)>
```

mixture-of-experts architecture.

Evaluation uses a local OpenAI-compatible SGLang endpoint together with Harbor and Daytona. Each run starts the model server, verifies readiness with a chat-completions request, generates a Harbor configuration with trajectory recording, and launches sandbox evaluation. The checkpoint queue evaluates base and fine-tuned Qwen3.5-27B and Qwen3.5-122B-A10B checkpoints on Terminal-Bench 2, Terminal-Bench Hard, and Long-Horizon Terminal Bench. The three benchmarks share one serving configuration but use benchmark-specific concurrency, turn limits, and timeout settings. Three evaluation lanes run on separate GPU nodes, with retry and cleanup handling for transient Harbor and Daytona failures. Synthesized-task rollouts use the same infrastructure with sharded task replicas, controlled sampling temperature, bounded interaction length, and node-specific Daytona credentials. DeepSeek-V4-Pro and GPT-5.6-Sol fixed-solver evaluation replaces the local model server with an OpenAI-compatible external-API proxy while preserving the same task materialization, Harbor execution, and Daytona verification protocol.